

FILE: .env.example

```
# Copy this file to .env and fill in real values. NEVER commit .env to version control.

# --- Market data ---
DATA_MODE=live

# --- Telegram alerts ---
TELEGRAM_BOT_TOKEN=your_telegram_bot_token_here
TELEGRAM_CHAT_ID=your_telegram_chat_id_here

# --- Model / pipeline ---
MODEL_PATH=output/models/xauusd_direction_model.joblib
SIGNAL_LOG_DB_PATH=data/signal_log.db
PIPELINE_N_CANDLES=300
```

FILE: README.md

```
# pytest cache directory #

This directory contains data from the pytest's cache plugin,
which provides the `--lf` and `--ff` options, as well as the `cache` fixture.

**Do not** commit this to version control.

See [the docs](https://docs.pytest.org/en/stable/how-to/cache.html) for more information.
```

FILE: alerts\\_\_init\_\_.py

```
# [EMPTY FILE]
```

FILE: alerts\formatter.py

```
"""
Formats a signal JSON dict into a human-readable Telegram message.
Pure formatting logic, no network calls, no side effects - kept separate from
telegram_bot.py so it is trivially unit-testable without mocking HTTP calls.
"""
from datetime import datetime, timezone

def format_signal_message(signal: dict) -> str:
    """
    signal is the dict returned by realtime/pipeline.py::generate_signal().
    Produces a clearly formatted, emoji-free (professional tone) message with
    bias, confidence, entry zone, invalidation, and target levels.
    """
    bias = signal["bias"]
    confidence_pct = round(signal["confidence"] * 100, 1)
    regime = signal.get("regime", "unknown")
    session = signal.get("session", "unknown")

    if bias == "no_trade":
        return (
            f"XAUUSD Signal Update\n"
            f"Bias: NO TRADE\n"
            f"Regime: {regime}\n"
            f"Session: {session}\n"
            f"Reason: {signal.get('reasoning_summary', 'Confidence below threshold')}"
        )

    direction_label = "LONG" if bias == "long" else "SHORT"
    entry_zone = signal.get("entry_zone")
    invalidation = signal.get("invalidation")
    targets = signal.get("targets") or []

    entry_zone_str = f"{entry_zone[0]} - {entry_zone[1]}" if entry_zone else "N/A"
    targets_str = ", ".join(str(t) for t in targets) if targets else "N/A"

    return (
        f"XAUUSD Signal Alert\n"
        f"Bias: {direction_label}\n"
        f"Confidence: {confidence_pct}%\n"
        f"Regime: {regime}\n"
        f"Session: {session}\n"
        f"Entry zone: {entry_zone_str}\n"
        f"Invalidation: {invalidation}\n"
        f"Target(s): {targets_str}\n"
        f"Reasoning: {signal.get('reasoning_summary', '')}"
    )
```

FILE: alerts\telegram\_bot.py

```
"""
Telegram bot integration - sends alerts ONLY for signals at/above the configured
confidence threshold (config.yaml alerts.telegram_min_confidence).

CRITICAL: bot token is read exclusively from environment variable TELEGRAM_BOT_TOKEN,
```

never hardcoded (see config/loader.py::get\_env). Chat ID is likewise env-based.

Cooldown and max-alerts-per-day logic is enforced here to satisfy the project's "fewer, higher-confidence alerts" design goal - even a correctly high-confidence signal is suppressed if it arrives within the cooldown window or exceeds the daily cap.

```
"""
import time
import requests
from datetime import datetime, timezone
from typing import Optional

from config.loader import get_env
from alerts.formatter import format_signal_message


class TelegramAlertBot:
    def __init__(self, cfg: dict, bot_token: str = None, chat_id: str = None):
        self.cfg = cfg
        self.bot_token = bot_token or get_env("TELEGRAM_BOT_TOKEN", required=True)
        self.chat_id = chat_id or get_env("TELEGRAM_CHAT_ID", required=True)
        self.base_url = f"https://api.telegram.org/bot{self.bot_token}/sendMessage"

        self._last_alert_ts: Optional[float] = None
        self._alerts_sent_today = 0
        self._current_day = datetime.now(timezone.utc).date()

    def _reset_daily_counter_if_needed(self):
        today = datetime.now(timezone.utc).date()
        if today != self._current_day:
            self._current_day = today
            self._alerts_sent_today = 0

    def _should_send(self, signal: dict) -> bool:
        """Gate on confidence threshold, cooldown, and daily cap - all from config.yaml."""
        alert_cfg = self.cfg["alerts"]

        if signal["bias"] == "no_trade":
            return False
        if signal["confidence"] < alert_cfg["telegram_min_confidence"]:
            return False

        self._reset_daily_counter_if_needed()
        if self._alerts_sent_today >= alert_cfg["max_alerts_per_day"]:
            return False

        if self._last_alert_ts is not None:
            elapsed_minutes = (time.time() - self._last_alert_ts) / 60.0
            if elapsed_minutes < alert_cfg["cooldown_minutes"]:
                return False

        return True

    def send_alert_if_qualified(self, signal: dict) -> bool:
        """
        Returns True if an alert was actually sent, False if suppressed.
        Never raises on suppression - suppression is expected, normal behavior.
        """
        if not self._should_send(signal):
            return False

        message = format_signal_message(signal)
        response = requests.post(self.base_url, data={"chat_id": self.chat_id, "text": message}, timeout=10)
        response.raise_for_status()

        self._last_alert_ts = time.time()
        self._alerts_sent_today += 1
        return True
```

FILE: alerts\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: alerts\tests\test\_formatter.py

```
"""
Unit tests for alerts/formatter.py and alerts/telegram_bot.py gating logic.
Network calls are mocked - no real Telegram API calls are made in tests.
Run with: pytest alerts/tests/test_formatter.py -v
"""
import os
import sys
import time
from unittest.mock import patch, MagicMock

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from alerts.formatter import format_signal_message
```

```

from alerts.telegram_bot import TelegramAlertBot

CFG = load_config()

def _sample_signal(bias="long", confidence=0.8):
    return {
        "bias": bias,
        "confidence": confidence,
        "entry_zone": [2400.0, 2400.5] if bias != "no_trade" else None,
        "invalidation": 2398.0 if bias != "no_trade" else None,
        "targets": [2403.0] if bias != "no_trade" else None,
        "reasoning_summary": "test reasoning",
        "regime": "trend_up",
        "session": "london",
        "timestamp_utc": 1700000000,
        "generated_at": "2026-07-24T08:00:00+00:00",
    }

def test_format_no_trade_message():
    signal = _sample_signal(bias="no_trade", confidence=0.0)
    msg = format_signal_message(signal)
    assert "NO TRADE" in msg
    assert "test reasoning" in msg

def test_format_long_message_contains_key_fields():
    signal = _sample_signal(bias="long", confidence=0.8)
    msg = format_signal_message(signal)
    assert "LONG" in msg
    assert "80.0%" in msg
    assert "2400.0" in msg
    assert "2398.0" in msg
    assert "2403.0" in msg

def test_format_short_message_direction_label():
    signal = _sample_signal(bias="short", confidence=0.75)
    msg = format_signal_message(signal)
    assert "SHORT" in msg

def test_bot_suppresses_no_trade_signal():
    bot = TelegramAlertBot(CFG, bot_token="fake_token", chat_id="fake_chat_id")
    signal = _sample_signal(bias="no_trade", confidence=0.0)
    assert bot._should_send(signal) is False

def test_bot_suppresses_below_confidence_threshold():
    bot = TelegramAlertBot(CFG, bot_token="fake_token", chat_id="fake_chat_id")
    low_conf_signal = _sample_signal(bias="long", confidence=0.1)
    assert bot._should_send(low_conf_signal) is False

def test_bot_allows_high_confidence_signal():
    bot = TelegramAlertBot(CFG, bot_token="fake_token", chat_id="fake_chat_id")
    high_conf_signal = _sample_signal(bias="long", confidence=0.95)
    assert bot._should_send(high_conf_signal) is True

def test_bot_enforces_cooldown():
    bot = TelegramAlertBot(CFG, bot_token="fake_token", chat_id="fake_chat_id")
    signal = _sample_signal(bias="long", confidence=0.95)
    bot._last_alert_ts = time.time() # simulate an alert just sent
    assert bot._should_send(signal) is False, "Cooldown period must suppress immediate re-alert"

def test_bot_enforces_daily_cap():
    bot = TelegramAlertBot(CFG, bot_token="fake_token", chat_id="fake_chat_id")
    bot._alerts_sent_today = CFG["alerts"]["max_alerts_per_day"]
    signal = _sample_signal(bias="long", confidence=0.95)
    assert bot._should_send(signal) is False, "Daily cap must suppress further alerts"

@patch("alerts.telegram_bot.requests.post")
def test_send_alert_if_qualified_calls_api_and_updates_state(mock_post):
    mock_response = MagicMock()
    mock_response.raise_for_status = MagicMock()
    mock_post.return_value = mock_response

    bot = TelegramAlertBot(CFG, bot_token="fake_token", chat_id="fake_chat_id")
    signal = _sample_signal(bias="long", confidence=0.95)

    sent = bot.send_alert_if_qualified(signal)
    assert sent is True
    assert mock_post.called

```

```

assert bot._alerts_sent_today == 1
assert bot._last_alert_ts is not None

```

```

@patch("alerts.telegram_bot.requests.post")
def test_send_alert_if_qualified_skips_api_call_when_suppressed(mock_post):
    bot = TelegramAlertBot(CFG, bot_token="fake_token", chat_id="fake_chat_id")
    signal = _sample_signal(bias="no_trade", confidence=0.0)

    sent = bot.send_alert_if_qualified(signal)
    assert sent is False
    assert not mock_post.called

```

FILE: backtest\\_\_init\_\_.py

```

# [EMPTY FILE]

```

FILE: backtest\engine.py

```

"""
Event-driven backtester for the rule-based baseline (and later, ML/ensemble) signals.

CRITICAL DESIGN DECISION - avoiding look-ahead in execution timing:
A signal decided from information available AT THE CLOSE of candle i can only be
acted upon starting at candle i+1 (we cannot retroactively fill an order inside the
candle whose close just produced the signal). This engine enforces that gap
explicitly: signal_at_close(i) -> entry_at_open(i+1). This is the second most common
source of backtest look-ahead bias after feature leakage, so it is isolated here.

The engine processes candles strictly in order (no vectorized shortcuts), maintains
at most ONE open position at a time (consistent with small-deposit, no-leverage-blowup
risk context), and applies spread/slippage on both entry and exit.
"""

import numpy as np
import pandas as pd
from dataclasses import dataclass, field
from typing import Optional, List

from regime.classifier import RegimeLabel

@dataclass
class Trade:
    entry_ts: int
    entry_price: float
    direction: int # +1 long, -1 short
    stop_price: float
    target_price: float
    session: str
    regime_at_entry: str
    exit_ts: Optional[int] = None
    exit_price: Optional[float] = None
    exit_reason: Optional[str] = None # "target", "stop", "timeout"
    pnl: Optional[float] = None

def rule_based_signal(regime: RegimeLabel) -> int:
    """
    Baseline directional bias purely from regime classification.
    +1 = long bias, -1 = short bias, 0 = no signal (no_trade/range/compression/reversal_watch).
    This is intentionally conservative per project risk context: only clear trend
    regimes generate a directional bias in the baseline; everything else is NO TRADE.
    """
    if regime == RegimeLabel.TREND_UP:
        return 1
    elif regime == RegimeLabel.TREND_DOWN:
        return -1
    return 0

class EventDrivenBacktester:
    """
    Processes a DataFrame of candles (with 'regime' column already attached) one
    row at a time, opening/closing at most one position, applying spread/slippage
    from config, and using the SAME +X/-Y/N barrier logic as labeling/ for exits -
    so backtest outcomes are directly comparable to the supervised learning target.
    """

    def __init__(self, cfg: dict):
        self.cfg = cfg
        bt_cfg = cfg["backtest"]
        lab_cfg = cfg["labeling"]
        self.spread = bt_cfg["spread_points"] / 100.0 # points -> USD price units (see config note)
        self.slippage = bt_cfg["slippage_points"] / 100.0
        self.balance = bt_cfg["initial_balance"]
        self.risk_pct = bt_cfg["risk_per_trade_pct"] / 100.0
        self.target_x = lab_cfg["target_pips_x"]
        self.stop_y = lab_cfg["stop_pips_y"]

```

```

self.horizon_n = lab_cfg["horizon_candles_n"]

self.trades: List[Trade] = []
self.equity_curve: List[float] = [self.balance]

def run(self, df: pd.DataFrame) -> List[Trade]:
    """
    df must contain: timestamp_utc, open, high, low, close, session, regime.
    Iterates candle-by-candle. At most one open position at a time.
    Signal decided at close of candle i; entry executed at open of candle i+1.
    """
    n = len(df)
    open_position: Optional[Trade] = None
    pending_signal: Optional[int] = None # signal decided at i, to be acted on at i+1

    opens = df["open"].values
    highs = df["high"].values
    lows = df["low"].values
    closes = df["close"].values
    timestamps = df["timestamp_utc"].values
    sessions = df["session"].values
    regimes = df["regime"].values

    for i in range(n):
        # 1. Execute any pending signal from the PREVIOUS candle's close, using THIS candle's open
        if open_position is None and pending_signal is not None and pending_signal != 0:
            direction = pending_signal
            entry_price = opens[i] + (self.spread / 2 if direction == 1 else -self.spread / 2)
            stop_price = entry_price - direction * self.stop_y
            target_price = entry_price + direction * self.target_x
            open_position = Trade(
                entry_ts=int(timestamps[i]),
                entry_price=entry_price,
                direction=direction,
                stop_price=stop_price,
                target_price=target_price,
                session=str(sessions[i]),
                regime_at_entry=str(regimes[i - 1]) if i > 0 else str(regimes[i]),
            )
            pending_signal = None

        # 2. If a position is open, check for target/stop hit using THIS candle's high/low
        if open_position is not None:
            direction = open_position.direction
            hit_target = (highs[i] >= open_position.target_price) if direction == 1 else (lows[i] <= open_position.target_price)
            hit_stop = (lows[i] <= open_position.stop_price) if direction == 1 else (highs[i] >= open_position.stop_price)

            candles_held = self._candles_since(df, open_position.entry_ts, timestamps[i])

            exit_reason = None
            exit_price = None
            if hit_target and hit_stop:
                # Conservative: ambiguous same-candle double touch -> assume stop hit first
                exit_reason, exit_price = "stop", open_position.stop_price
            elif hit_stop:
                exit_reason, exit_price = "stop", open_position.stop_price
            elif hit_target:
                exit_reason, exit_price = "target", open_position.target_price
            elif candles_held >= self.horizon_n:
                exit_reason, exit_price = "timeout", closes[i]

            if exit_reason is not None:
                exit_price += (-self.slippage if direction == 1 else self.slippage)
                open_position.exit_ts = int(timestamps[i])
                open_position.exit_price = exit_price
                open_position.exit_reason = exit_reason
                open_position.pnl = direction * (exit_price - open_position.entry_price)
                self.balance += open_position.pnl
                self.equity_curve.append(self.balance)
                self.trades.append(open_position)
                open_position = None

        # 3. Decide signal AT THE CLOSE of this candle, for execution at i+1 (never at i - that's look-ahead)
        if open_position is None:
            pending_signal = rule_based_signal(regimes[i])

    return self.trades

@staticmethod
def _candles_since(df: pd.DataFrame, entry_ts: int, current_ts: int) -> int:
    """Count candles elapsed between entry and current timestamp using the DataFrame's own spacing."""
    step = df["timestamp_utc"].diff().mode().iloc[0] if len(df) > 1 else 1
    return int((current_ts - entry_ts) / step)

```

FILE: backtest\metrics.py

```

"""
Backtest performance metrics: win rate, profit factor, max drawdown - computed

```

```

overall and split per trading session (Asia/London/NY).
"""
import numpy as np
import pandas as pd
from typing import List
from backtest.engine import Trade

def trades_to_dataframe(trades: List[Trade]) -> pd.DataFrame:
    if not trades:
        return pd.DataFrame(columns=["entry_ts", "exit_ts", "direction", "session", "regime_at_entry", "pnl", "exit_reason"])
    return pd.DataFrame([
        {
            "entry_ts": t.entry_ts, "exit_ts": t.exit_ts, "direction": t.direction,
            "session": t.session, "regime_at_entry": t.regime_at_entry,
            "pnl": t.pnl, "exit_reason": t.exit_reason,
        } for t in trades])

def compute_metrics(trades_df: pd.DataFrame) -> dict:
    """
    win_rate: % of trades with pnl > 0
    profit_factor: gross profit / gross loss (inf if no losses, NaN if no trades)
    max_drawdown: largest peak-to-trough decline in cumulative pnl equity curve
    """
    if len(trades_df) == 0:
        return {"n_trades": 0, "win_rate": np.nan, "profit_factor": np.nan, "max_drawdown": np.nan, "total_pnl": 0.0}

    wins = trades_df[trades_df["pnl"] > 0]
    losses = trades_df[trades_df["pnl"] <= 0]
    gross_profit = wins["pnl"].sum()
    gross_loss = -losses["pnl"].sum()

    win_rate = len(wins) / len(trades_df) * 100
    profit_factor = gross_profit / gross_loss if gross_loss > 0 else np.inf

    cum_pnl = trades_df["pnl"].cumsum()
    running_max = cum_pnl.cummax()
    drawdown = cum_pnl - running_max
    max_drawdown = drawdown.min() if len(drawdown) > 0 else 0.0

    return {
        "n_trades": len(trades_df),
        "win_rate": win_rate,
        "profit_factor": profit_factor,
        "max_drawdown": max_drawdown,
        "total_pnl": trades_df["pnl"].sum(),
    }

def compute_metrics_per_session(trades_df: pd.DataFrame) -> dict:
    """
    Splits metrics by session tag. A trade's session is tagged at ENTRY time
    (open_position.session, set from the candle where entry executed) - consistent
    with how a live trader would categorize "which session did I take this trade in".
    """
    if len(trades_df) == 0:
        return {}
    results = {}
    for session_name, group in trades_df.groupby("session"):
        results[session_name] = compute_metrics(group)
    return results

```

FILE: backtest\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: backtest\tests\test\_engine.py

```

"""
Unit tests for backtest/engine.py, metrics.py, and walk_forward.py.
Run with: pytest backtest/tests/test_engine.py -v
"""
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import fetch_mock_candles
from features.indicators import build_all_indicators
from regime.classifier import add_regime_indicators, classify_regime_series, RegimeLabel
from backtest.engine import EventDrivenBacktester, rule_based_signal, Trade
from backtest.metrics import trades_to_dataframe, compute_metrics, compute_metrics_per_session
from backtest.walk_forward import generate_windows, run_walk_forward

CFG = load_config()

```

```
SESSIONS = CFG["sessions"]
```

```
def _prepared_df(n=800, seed=42):
    df = fetch_mock_candles("M15", n_candles=n, sessions_config=SESSIONS, seed=seed)
    df = build_all_indicators(df, CFG)
    df = add_regime_indicators(df, CFG)
    df["regime"] = classify_regime_series(df, CFG)
    return df
```

```
def test_rule_based_signal_mapping():
    assert rule_based_signal(RegimeLabel.TREND_UP) == 1
    assert rule_based_signal(RegimeLabel.TREND_DOWN) == -1
    assert rule_based_signal(RegimeLabel.RANGE) == 0
    assert rule_based_signal(RegimeLabel.NO_TRADE) == 0
    assert rule_based_signal(RegimeLabel.COMPRESSION) == 0
    assert rule_based_signal(RegimeLabel.REVERSAL_WATCH) == 0
```

```
def test_engine_produces_trades_with_valid_fields():
    df = _prepared_df()
    engine = EventDrivenBacktester(CFG)
    trades = engine.run(df)
    assert isinstance(trades, list)
    for t in trades:
        assert isinstance(t, Trade)
        assert t.exit_reason in ("target", "stop", "timeout")
        assert t.pnl is not None
        assert t.exit_ts > t.entry_ts # exit must happen strictly after entry
```

```
def test_entry_executes_one_candle_after_signal_no_lookahead():
    """
    CRITICAL TEST: manually verify that a trade's entry_price is derived from the
    OPEN of the candle AFTER the regime signal changed, never the same candle.
    """
    df = _prepared_df()
    engine = EventDrivenBacktester(CFG)
    trades = engine.run(df)
    ts_to_idx = {int(ts): i for i, ts in enumerate(df["timestamp_utc"].values)}
    for t in trades:
        entry_idx = ts_to_idx[t.entry_ts]
        assert entry_idx > 0, "Entry cannot happen on the very first candle (no prior signal possible)"
        # The regime that triggered this trade must have been observed on a PRIOR candle
        prior_regime = df["regime"].iloc[entry_idx - 1]
        signal = rule_based_signal(prior_regime)
        assert signal == t.direction, "Entry direction must match the signal decided on the PRIOR candle's close"
```

```
def test_at_most_one_open_position_at_a_time():
    df = _prepared_df()
    engine = EventDrivenBacktester(CFG)
    trades = engine.run(df)
    trades_sorted = sorted(trades, key=lambda t: t.entry_ts)
    for i in range(1, len(trades_sorted)):
        assert trades_sorted[i].entry_ts >= trades_sorted[i - 1].exit_ts, \
            "New trade must not open before the previous one closed"
```

```
def test_metrics_computation_basic():
    df = _prepared_df()
    engine = EventDrivenBacktester(CFG)
    trades = engine.run(df)
    trades_df = trades_to_dataframe(trades)
    metrics = compute_metrics(trades_df)
    assert metrics["n_trades"] == len(trades)
    if metrics["n_trades"] > 0:
        assert 0 <= metrics["win_rate"] <= 100
        assert metrics["max_drawdown"] <= 0
```

```
def test_metrics_per_session_keys_are_valid():
    df = _prepared_df()
    engine = EventDrivenBacktester(CFG)
    trades = engine.run(df)
    trades_df = trades_to_dataframe(trades)
    per_session = compute_metrics_per_session(trades_df)
    for session_name, m in per_session.items():
        assert m["n_trades"] > 0
```

```
def test_walk_forward_windows_never_overlap_train_into_test():
    df = _prepared_df(n=3000)
    windows = generate_windows(df, train_window_days=5, test_window_days=2, step_days=2)
    for w in windows:
        assert w.train_end_ts <= w.test_start_ts, "Train window must never extend into test window"
```

```

        assert w.test_start_ts < w.test_end_ts

def test_walk_forward_runner_calls_strategy_per_fold():
    df = _prepared_df(n=3000)

    def dummy_strategy(train_df, test_df, cfg):
        engine = EventDrivenBacktester(cfg)
        trades = engine.run(test_df.reset_index(drop=True))
        trades_df = trades_to_dataframe(trades)
        return compute_metrics(trades_df)

    # Use shorter windows for the test to guarantee at least one fold with limited mock data
    test_cfg = {**CFG, "backtest": {**CFG["backtest"], "walk_forward": {"train_window_days": 5, "test_window_days": 2, "step_days": 2}}}
    results = run_walk_forward(df, test_cfg, dummy_strategy)
    assert isinstance(results, list)
    for r in results:
        assert "window" in r
        assert "n_trades" in r

```

FILE: backtest\walk\_forward.py

```

"""
Walk-forward validation: rolling train/test windows over time, re-evaluating the
strategy on each out-of-sample test window without ever training on future data
relative to that window.

For the rule-based baseline (Step 7), there is no "training" step per se, but the
walk-forward harness is built now so it works identically once model/ (Step 8+)
introduces an actual trained model that MUST be fit only on the train window and
evaluated only on the immediately following test window - never on data that
overlaps or precedes what was used for fitting.
"""

import pandas as pd
from typing import Callable, List, Dict
from dataclasses import dataclass

@dataclass
class WalkForwardWindow:
    train_start_ts: int
    train_end_ts: int
    test_start_ts: int
    test_end_ts: int

def generate_windows(df: pd.DataFrame, train_window_days: int, test_window_days: int,
                    step_days: int) -> List[WalkForwardWindow]:
    """
    Generate non-overlapping (in test) rolling windows across the full timestamp range.
    Each test window strictly follows its train window in time - train_end_ts <= test_start_ts,
    guaranteeing no test-window information can leak backward into training.
    """
    if len(df) == 0:
        return []

    min_ts = int(df["timestamp_utc"].min())
    max_ts = int(df["timestamp_utc"].max())
    day_s = 86400

    windows = []
    train_start = min_ts
    while True:
        train_end = train_start + train_window_days * day_s
        test_start = train_end
        test_end = test_start + test_window_days * day_s
        if test_end > max_ts:
            break
        windows.append(WalkForwardWindow(train_start, train_end, test_start, test_end))
        train_start += step_days * day_s

    return windows

def run_walk_forward(df: pd.DataFrame, cfg: dict,
                    strategy_fn: Callable[[pd.DataFrame, pd.DataFrame, dict], Dict]) -> List[dict]:
    """
    strategy_fn(train_df, test_df, cfg) -> metrics dict for that fold.
    For the rule-based baseline, train_df is unused (no fitting needed) but is still
    passed for interface consistency with the future ML strategy_fn.
    Returns a list of per-fold results, each tagged with its window boundaries.
    """
    wf_cfg = cfg["backtest"]["walk_forward"]
    windows = generate_windows(df, wf_cfg["train_window_days"], wf_cfg["test_window_days"], wf_cfg["step_days"])

    results = []
    for w in windows:
        train_df = df[(df["timestamp_utc"] >= w.train_start_ts) & (df["timestamp_utc"] < w.train_end_ts)]

```



```

test_df = df[(df["timestamp_utc"] >= w.test_start_ts) & (df["timestamp_utc"] < w.test_end_ts)]
if len(test_df) == 0:
    continue
fold_result = strategy_fn(train_df, test_df, cfg)
fold_result["window"] = w
results.append(fold_result)

return results

```

FILE: config\\_\_init\_\_.py

# [EMPTY FILE]

FILE: config\config.yaml

```

ensemble:
  min_confidence_to_alert: 0.60
  min_regime_confidence: 0.75
  suppress_regimes:
    - range
    - compression
    - reversal_watch
  rule_weight: 0.55
  ml_weight: 0.45

market_data:
  provider: mt5
  timeframe: M15
  closed_candles_only: true

general:
  db_path: data/market_data_mt5.sqlite

assets:
  XAUUSD:
    enabled: true
    mt5_symbol: GOLD
    display_name: XAU/USD
    model_path: output/models/xauusd_direction_model.joblib

  XAGUSD:
    enabled: true
    mt5_symbol: SILVER
    display_name: XAG/USD
    model_path: output/models/xagusd_direction_model.joblib

  BTCUSD:
    enabled: true
    mt5_symbol: BITCOIN
    display_name: BTC/USD
    model_path: output/models/btcusd_direction_model.joblib

  ETHUSD:
    enabled: true
    mt5_symbol: ETHEREUM
    display_name: ETH/USD
    model_path: output/models/ethusd_direction_model.joblib

  EURUSD:
    enabled: true
    mt5_symbol: EURUSD
    display_name: EUR/USD
    model_path: output/models/eurusd_direction_model.joblib

  GBPUSD:
    enabled: true
    mt5_symbol: GBPUSD
    display_name: GBP/USD
    model_path: output/models/gbpusd_direction_model.joblib

alerts:
  cooldown_minutes: 60
  max_alerts_per_day: 6

features:
  ema_periods: [9, 21, 50, 200]
  rsi_period: 14
  macd:
    fast: 12
    slow: 26
    signal: 9
  atr_period: 14
  bollinger:
    period: 20
    std_dev: 2.0
  structure_lookback: 20

regime:

```

```

min_candles_for_regime: 50
no_trade_volatility_floor: 0.0005
atr_spike_multiplier: 1.8
bb_width_compression_pctile: 20
adx_trend_threshold: 25

model:
  random_seed: 42
  calibration_method: sigmoid
  train_ratio: 0.8

labeling:
  method: atr_scaled
  horizon_candles_n: 32
  target_atr_multiplier: 1.2
  stop_atr_multiplier: 0.9
  atr_column: atr
  target_pips_x: 15.0
  stop_pips_y: 10.0

sessions:
  asia:
    start: 0
    end: 8
  london:
    start: 8
    end: 13
  newyork:
    start: 13
    end: 22

backtest:
  spread_points: 30
  slippage_points: 10
  initial_balance: 10000.0
  risk_per_trade_pct: 1.0
  walk_forward:
    train_window_days: 180
    test_window_days: 30
    step_days: 30

```

FILE: config\loader.py

```

"""
Config loader utility - shared across all modules.
Ensures a single source of truth: config/config.yaml.
"""
import os
import yaml

_CONFIG_CACHE = None

def load_config(path: str = None) -> dict:
    """
    Load and cache the master YAML config.
    Path defaults to config/config.yaml relative to project root.
    """
    global _CONFIG_CACHE
    if _CONFIG_CACHE is not None:
        return _CONFIG_CACHE

    if path is None:
        # Resolve relative to this file's directory (config/)
        path = os.path.join(os.path.dirname(__file__), "config.yaml")

    with open(path, "r") as f:
        _CONFIG_CACHE = yaml.safe_load(f)
    return _CONFIG_CACHE

def get_env(key: str, default=None, required: bool = False):
    """
    Fetch a secret/config value from environment variables.
    Never hardcode API keys or tokens - always route through this function.
    """
    val = os.environ.get(key, default)
    if required and val is None:
        raise EnvironmentError(f"Required environment variable '{key}' is not set.")
    return val

```

FILE: data\\_\_init\_\_.py

```
# [EMPTY FILE]
```

FILE: data\ingestion.py

```

import time
import numpy as np

```

```

import requests
import pandas as pd

from config.loader import get_env
from data.session_tagger import tag_dataframe

TIMEFRAME_TO_SECONDS = {
    "M1": 60,
    "M5": 300,
    "M15": 900,
    "H1": 3600,
    "H4": 14400,
}

TWELVE_DATA_MAX_OUTPUTSIZE = 5000
_MIN_SECONDS_BETWEEN_REQUESTS = 60 / 8 # 8 requests/minute free-tier limit

def fetch_mock_candles(timeframe: str, n_candles: int, sessions_config: dict,
                       end_ts: int = None, seed: int = 42) -> pd.DataFrame:
    """
    Generate a deterministic, reproducible synthetic OHLCV series for offline testing.
    Uses a random walk around a plausible XAUUSD price level (~2000-2600 range as of 2026).
    This is ONLY for testing the pipeline shape - never used for real signal generation.
    """
    if timeframe not in TIMEFRAME_TO_SECONDS:
        raise ValueError(f"Unsupported timeframe: {timeframe}")

    rng = np.random.default_rng(seed)
    step_seconds = TIMEFRAME_TO_SECONDS[timeframe]
    if end_ts is None:
        end_ts = int(time.time())
    end_ts = (end_ts // step_seconds) * step_seconds

    timestamps = [end_ts - i * step_seconds for i in range(n_candles)][::-1]

    price = 2400.0
    rows = []
    for ts in timestamps:
        drift = rng.normal(0, 1.5)
        open_p = price
        close_p = open_p + drift
        high_p = max(open_p, close_p) + abs(rng.normal(0, 1.0))
        low_p = min(open_p, close_p) - abs(rng.normal(0, 1.0))
        vol = abs(rng.normal(100, 20))
        rows.append([ts, open_p, high_p, low_p, close_p, vol])
        price = close_p

    df = pd.DataFrame(rows, columns=["timestamp_utc", "open", "high", "low", "close", "volume"])
    df = tag_dataframe(df, sessions_config)
    return df

def _request_with_backoff(url: str, params: dict, max_retries: int = 5) -> dict:
    """Perform a GET request with exponential backoff on 429s and API-level rate limit errors."""
    delay = 2
    for attempt in range(max_retries):
        response = requests.get(url, params=params, timeout=60)
        if response.status_code == 429:
            time.sleep(delay)
            delay *= 2
            continue
        data = response.json()
        if isinstance(data, dict) and data.get("code") == 429:
            time.sleep(delay)
            delay *= 2
            continue
        response.raise_for_status()
        return data
    raise RuntimeError(f"Exceeded max retries ({max_retries}) due to rate limiting.")

def fetch_live_candles(timeframe: str, n_candles: int, sessions_config: dict,
                       api_key_env: str = "TWELVE_DATA_API_KEY",
                       base_url: str = "https://api.twelvedata.com/time_series") -> pd.DataFrame:
    """
    Fetch real OHLCV candles from a market data vendor.
    Requires API key set via environment variable (never hardcoded).
    """
    api_key = get_env(api_key_env, required=True)

    interval_map = {"M1": "1min", "M5": "5min", "M15": "15min", "H1": "1h", "H4": "4h"}
    if timeframe not in interval_map:
        raise ValueError(f"Unsupported timeframe: {timeframe}")

    if n_candles > TWELVE_DATA_MAX_OUTPUTSIZE:

```

```

        raise ValueError(f"n_candles exceeds Twelve Data max outputsize of {TWELVE_DATA_MAX_OUTPUTSIZE}")

params = {
    "symbol": "XAU/USD",
    "interval": interval_map[timeframe],
    "outputsize": n_candles,
    "apikey": api_key,
    "timezone": "UTC",
    "format": "JSON",
}

resp = requests.get(base_url, params=params, timeout=60)
resp.raise_for_status()
payload = resp.json()

if "values" not in payload:
    raise RuntimeError(f"Unexpected API response, no 'values' key: {payload}")

rows = []
for item in payload["values"]:
    ts = int(pd.Timestamp(item["datetime"], tz="UTC").timestamp())
    rows.append([
        ts,
        float(item["open"]),
        float(item["high"]),
        float(item["low"]),
        float(item["close"]),
        float(item.get("volume", 0.0) or 0.0),
    ])

df = pd.DataFrame(rows, columns=["timestamp_utc", "open", "high", "low", "close", "volume"])
df = df.sort_values("timestamp_utc").reset_index(drop=True)
df = tag_dataframe(df, sessions_config)
return df


def backfill_historical(timeframe: str, start_date: str, end_date: str, sessions_config: dict,
                        api_key_env: str = "TWELVE_DATA_API_KEY",
                        base_url: str = "https://api.twelvedata.com/time_series") -> pd.DataFrame:
    """Pull historical candles in 500-row chunks (free-tier safe)."""
    import time as _time
    api_key = get_env(api_key_env, required=True)
    interval_map = {"M1": "1min", "M5": "5min", "M15": "15min", "H1": "1h", "H4": "4h"}
    if timeframe not in interval_map:
        raise ValueError("Unsupported timeframe: " + timeframe)
    CHUNK_SIZE = 500
    all_rows = []
    current_end = end_date
    while True:
        params = {
            "symbol": "XAU/USD",
            "interval": interval_map[timeframe],
            "outputsize": CHUNK_SIZE,
            "end_date": current_end,
            "apikey": api_key,
            "timezone": "UTC",
            "format": "JSON",
        }
        data = _request_with_backoff(base_url, params)
        values = data.get("values", [])
        if not values:
            break
        values = [v for v in values if v["datetime"][:10] >= start_date]
        if not values:
            break
        all_rows.extend(values)
        oldest_ts = values[-1]["datetime"]
        if oldest_ts[:10] <= start_date:
            break
        current_end = oldest_ts
        _time.sleep(8)
    print()
    if not all_rows:
        raise RuntimeError("No data returned for " + timeframe + " between " + start_date + " and " + end_date)
    df = pd.DataFrame(all_rows)
    df["datetime"] = pd.to_datetime(df["datetime"])
    df = df.drop_duplicates(subset="datetime").sort_values("datetime").reset_index(drop=True)
    for col in ["open", "high", "low", "close"]:
        df[col] = df[col].astype(float)
    df["volume"] = pd.to_numeric(df["volume"], errors="coerce").fillna(0.0) if "volume" in df.columns else 0.0
    df["timestamp_utc"] = df["datetime"].astype("int64") // 10**9
    df = df.drop(columns=["datetime"])
    df = df.drop_duplicates(subset="timestamp_utc").sort_values("timestamp_utc").reset_index(drop=True)
    df = tag_dataframe(df, sessions_config)
    return df

```

```

def fetch_candles(timeframe: str, n_candles: int, sessions_config: dict,
                  mode: str = "mock", **kwargs) -> pd.DataFrame:
    """
    Unified entry point used by downstream code.
    mode="mock" for offline/dev/test, mode="live" for real API pull.
    """
    if mode == "mock":
        return fetch_mock_candles(timeframe, n_candles, sessions_config, **kwargs)
    elif mode == "live":
        return fetch_live_candles(timeframe, n_candles, sessions_config, **kwargs)
    else:
        raise ValueError(f"Unknown ingestion mode: {mode}")

```

FILE: data\mt5\_provider.before\_symbol\_fix.py

```

from __future__ import annotations

from datetime import datetime, timezone
from typing import Final

import MetaTrader5 as mt5
import pandas as pd

_TIMEFRAMES: Final[dict[str, int]] = {
    "M1": mt5.TIMEFRAME_M1,
    "M5": mt5.TIMEFRAME_M5,
    "M15": mt5.TIMEFRAME_M15,
    "M30": mt5.TIMEFRAME_M30,
    "H1": mt5.TIMEFRAME_H1,
    "H4": mt5.TIMEFRAME_H4,
    "D1": mt5.TIMEFRAME_D1,
}

class MT5ProviderError(RuntimeError):
    """Raised when FxPro MT5 data cannot be read safely."""

def initialize_mt5() -> None:
    if mt5.initialize():
        return
    raise MT5ProviderError(f"MT5 initialize failed: {mt5.last_error()}")

def shutdown_mt5() -> None:
    mt5.shutdown()

def validate_symbol(symbol: str) -> None:
    info = mt5.symbol_info(symbol)

    if info is None:
        raise MT5ProviderError(
            f"MT5 symbol '{symbol}' was not found. "
            "Keep FxPro MT5 open and show it in Market Watch."
        )

    if not info.visible and not mt5.symbol_select(symbol, True):
        raise MT5ProviderError(
            f"Could not select MT5 symbol '{symbol}': {mt5.last_error()}"
        )

def _normalize_rates(rates) -> pd.DataFrame:
    if rates is None or len(rates) == 0:
        raise MT5ProviderError(f"MT5 returned no candle data: {mt5.last_error()}")

    df = pd.DataFrame(rates)
    df["timestamp"] = pd.to_datetime(df["time"], unit="s", utc=True)
    df = df.rename(columns={"tick_volume": "volume"})

    required = ["timestamp", "open", "high", "low", "close", "volume"]
    df = df[required].copy()

    for column in ("open", "high", "low", "close", "volume"):
        df[column] = pd.to_numeric(df[column], errors="coerce")

    df = (
        df.dropna(subset=required)
        .drop_duplicates(subset=["timestamp"])
        .sort_values("timestamp")
        .reset_index(drop=True)
    )

    if df.empty:
        raise MT5ProviderError("MT5 returned candle rows, but none were valid")

```

```

        return df

def fetch_closed_candles(
    symbol: str,
    timeframe: str,
    count: int,
) -> pd.DataFrame:
    """Fetch only completed bars; MT5 position 0 is intentionally excluded."""
    if timeframe not in _TIMEFRAMES:
        raise ValueError(f"Unsupported timeframe: {timeframe}")

    if count < 1:
        raise ValueError("count must be at least 1")

    validate_symbol(symbol)

    rates = mt5.copy_rates_from_pos(
        symbol,
        _TIMEFRAMES[timeframe],
        1,
        count,
    )

    return _normalize_rates(rates)

def fetch_candles_range(
    symbol: str,
    timeframe: str,
    start_utc: datetime,
    end_utc: datetime,
) -> pd.DataFrame:
    """Fetch MT5 bars in a UTC range, excluding a currently forming bar."""
    if timeframe not in _TIMEFRAMES:
        raise ValueError(f"Unsupported timeframe: {timeframe}")

    if start_utc.tzinfo is None or end_utc.tzinfo is None:
        raise ValueError("start_utc and end_utc must be timezone-aware")

    if end_utc <= start_utc:
        raise ValueError("end_utc must be later than start_utc")

    validate_symbol(symbol)

    start_utc = start_utc.astimezone(timezone.utc)
    end_utc = end_utc.astimezone(timezone.utc)

    rates = mt5.copy_rates_range(
        symbol,
        _TIMEFRAMES[timeframe],
        start_utc,
        end_utc,
    )

    df = _normalize_rates(rates)

    now_utc = pd.Timestamp.now(tz="UTC")
    tf_minutes = {
        "M1": 1, "M5": 5, "M15": 15, "M30": 30,
        "H1": 60, "H4": 240, "D1": 1440,
    }[timeframe]

    current_bar_open = now_utc.floor(f"{tf_minutes}min")
    return df.loc[df["timestamp"] < current_bar_open].reset_index(drop=True)

```

FILE: data\mt5\_provider.py

```

from __future__ import annotations

from datetime import datetime, timezone
from typing import Final

import MetaTrader5 as mt5
import pandas as pd

```

```

_TIMEFRAMES: Final[dict[str, int]] = {
    "M1": mt5.TIMEFRAME_M1,
    "M5": mt5.TIMEFRAME_M5,
    "M15": mt5.TIMEFRAME_M15,
    "M30": mt5.TIMEFRAME_M30,
    "H1": mt5.TIMEFRAME_H1,
    "H4": mt5.TIMEFRAME_H4,
    "D1": mt5.TIMEFRAME_D1,
}

```

```

class MT5ProviderError(RuntimeError):
    """Raised when FxPro MT5 data cannot be read safely."""

def initialize_mt5() -> None:
    if mt5.initialize():
        return
    raise MT5ProviderError(f"MT5 initialize failed: {mt5.last_error()}")

def shutdown_mt5() -> None:
    mt5.shutdown()

def validate_symbol(symbol: str) -> None:
    """Confirm an MT5 symbol exists and enable it in Market Watch."""
    initialize_mt5()

    info = mt5.symbol_info(symbol)
    if info is None:
        raise MT5ProviderError(
            f"MT5 symbol {symbol!r} was not found. "
            "Check the exact name in the FxPro MT5 Symbols window."
        )

    if not mt5.symbol_select(symbol, True):
        code, message = mt5.last_error()
        raise MT5ProviderError(
            f"Could not enable MT5 symbol {symbol!r} in Market Watch: "
            f"{code} {message}"
        )

def _normalize_rates(rates) -> pd.DataFrame:
    if rates is None or len(rates) == 0:
        raise MT5ProviderError(f"MT5 returned no candle data: {mt5.last_error()}")

    df = pd.DataFrame(rates)
    df["timestamp"] = pd.to_datetime(df["time"], unit="s", utc=True)
    df = df.rename(columns={"tick_volume": "volume"})

    required = ["timestamp", "open", "high", "low", "close", "volume"]
    df = df[required].copy()

    for column in ("open", "high", "low", "close", "volume"):
        df[column] = pd.to_numeric(df[column], errors="coerce")

    df = (
        df.dropna(subset=required)
        .drop_duplicates(subset=["timestamp"])
        .sort_values("timestamp")
        .reset_index(drop=True)
    )

    if df.empty:
        raise MT5ProviderError("MT5 returned candle rows, but none were valid")

    return df

def fetch_closed_candles(
    symbol: str,
    timeframe: str,
    count: int,
) -> pd.DataFrame:
    """Fetch only completed bars; MT5 position 0 is intentionally excluded."""
    if timeframe not in _TIMEFRAMES:
        raise ValueError(f"Unsupported timeframe: {timeframe}")

    if count < 1:
        raise ValueError("count must be at least 1")

    validate_symbol(symbol)

    rates = mt5.copy_rates_from_pos(
        symbol,
        _TIMEFRAMES[timeframe],
        1,
        count,
    )

    return _normalize_rates(rates)

def fetch_candles_range(
    symbol: str,
    timeframe: str,
    start_utc: datetime,

```

```

        end_utc: datetime,
    ) -> pd.DataFrame:
        """Fetch MT5 bars in a UTC range, excluding a currently forming bar."""
        if timeframe not in _TIMEFRAMES:
            raise ValueError(f"Unsupported timeframe: {timeframe}")

        if start_utc.tzinfo is None or end_utc.tzinfo is None:
            raise ValueError("start_utc and end_utc must be timezone-aware")

        if end_utc <= start_utc:
            raise ValueError("end_utc must be later than start_utc")

        validate_symbol(symbol)

        start_utc = start_utc.astimezone(timezone.utc)
        end_utc = end_utc.astimezone(timezone.utc)

        rates = mt5.copy_rates_range(
            symbol,
            _TIMEFRAMES[timeframe],
            start_utc,
            end_utc,
        )

        df = _normalize_rates(rates)

        now_utc = pd.Timestamp.now(tz="UTC")
        tf_minutes = {
            "M1": 1, "M5": 5, "M15": 15, "M30": 30,
            "H1": 60, "H4": 240, "D1": 1440,
        }[timeframe]

        current_bar_open = now_utc.floor(f"{tf_minutes}min")
        return df.loc[df["timestamp"] < current_bar_open].reset_index(drop=True)

```

FILE: data\session\_tagger.py

```

"""
Session tagging based on UTC hour-of-day.
Session windows are read from config.yaml, not hardcoded here.
A candle can belong to multiple overlapping sessions (e.g. London+NY overlap);
in that case we tag with a combined label so backtest/metrics.py can still
report clean per-session and per-overlap breakdowns.
"""
import pandas as pd

def _hour_in_window(hour: int, start: int, end: int) -> bool:
    """Handle windows that do not wrap midnight (all configured windows here don't)."""
    return start <= hour < end

def tag_session(utc_timestamp, sessions_config: dict) -> str:
    """
    Given a UTC epoch-second timestamp (or pandas Timestamp) and the sessions
    sub-dict from config.yaml, return a session label string.
    """
    if isinstance(utc_timestamp, (int, float)):
        ts = pd.to_datetime(utc_timestamp, unit="s", utc=True)
    else:
        ts = pd.Timestamp(utc_timestamp)
        if ts.tzinfo is None:
            ts = ts.tz_localize("UTC")

    hour = ts.hour
    active = []
    for name, window in sessions_config.items():
        if _hour_in_window(hour, window["start"], window["end"]):
            active.append(name)

    if not active:
        return "off_session"
    return "_".join(sorted(active))

def tag_dataframe(df: pd.DataFrame, sessions_config: dict, ts_col: str = "timestamp_utc") -> pd.DataFrame:
    """Vectorized-ish session tagging applied row-wise (fine for daily ingestion batch sizes)."""
    df = df.copy()
    df["session"] = df[ts_col].apply(lambda t: tag_session(t, sessions_config))
    return df

```

FILE: data\signal\_log.py

```

"""
SQLite persistence layer for generated signals (audit trail of every signal the
system produced, whether or not it triggered a Telegram alert).

Design mirrors data/storage.py's pattern: single table, UTC epoch timestamps,

```



idempotent upserts keyed on timestamp\_utc so re-running the scheduler for the same candle never creates duplicate rows.

This is critical for accountability: the project brief requires transparency into WHY a signal was or wasn't sent, not just the alerts that went out. Every row here answers "what did the system think at time T", regardless of alert\_sent status.

```
"""
import sqlite3
import os
import json
import pandas as pd

TABLE_NAME = "signal_log"

def get_connection(db_path: str) -> sqlite3.Connection:
    os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA journal_mode=WAL;")
    return conn

def init_schema(db_path: str):
    conn = get_connection(db_path)
    try:
        conn.execute(f"""
            CREATE TABLE IF NOT EXISTS {TABLE_NAME} (
                timestamp_utc INTEGER PRIMARY KEY,
                generated_at TEXT NOT NULL,
                bias TEXT NOT NULL,
                confidence REAL NOT NULL,
                regime TEXT NOT NULL,
                session TEXT NOT NULL,
                entry_zone TEXT,
                invalidation REAL,
                targets TEXT,
                reasoning_summary TEXT,
                alert_sent INTEGER NOT NULL DEFAULT 0
            );
        """)
        conn.execute(f"CREATE INDEX IF NOT EXISTS idx_{TABLE_NAME}_ts ON {TABLE_NAME}(timestamp_utc);")
        conn.commit()
    finally:
        conn.close()

def log_signal(db_path: str, signal: dict, alert_sent: bool):
    """
    Persists one signal dict (the exact JSON shape returned by
    realtime/pipeline.py::generate_signal) plus whether an alert was actually sent.
    Uses INSERT OR REPLACE so re-scoring the same candle (e.g. after a restart)
    is idempotent, not a duplicate audit entry.
    """
    conn = get_connection(db_path)
    try:
        conn.execute(
            f"""INSERT OR REPLACE INTO {TABLE_NAME}
                (timestamp_utc, generated_at, bias, confidence, regime, session,
                 entry_zone, invalidation, targets, reasoning_summary, alert_sent)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (
                signal["timestamp_utc"],
                signal["generated_at"],
                signal["bias"],
                signal["confidence"],
                signal["regime"],
                signal["session"],
                json.dumps(signal["entry_zone"]) if signal.get("entry_zone") else None,
                signal.get("invalidation"),
                json.dumps(signal["targets"]) if signal.get("targets") else None,
                signal.get("reasoning_summary", ""),
                int(alert_sent),
            ),
        )
        conn.commit()
    finally:
        conn.close()

def read_signal_history(db_path: str, start_ts: int = None, end_ts: int = None) -> pd.DataFrame:
    """Read logged signals for auditing/reporting, sorted ascending by timestamp."""
    conn = get_connection(db_path)
    try:
        query = f"SELECT * FROM {TABLE_NAME}"
        clauses, params = [], []
        if start_ts is not None:
            clauses.append("timestamp_utc >= ?")

```

```

        params.append(start_ts)
    if end_ts is not None:
        clauses.append("timestamp_utc <= ?")
        params.append(end_ts)
    if clauses:
        query += " WHERE " + " AND ".join(clauses)
    query += " ORDER BY timestamp_utc ASC"
    df = pd.read_sql_query(query, conn, params=params)
finally:
    conn.close()
return df

```

FILE: data\storage.py

```

"""
SQLite storage for multi-asset OHLCV candles.

Each timeframe has its own table. Within each table, candles are uniquely
identified by (symbol, timestamp_utc), allowing multiple assets to share an
M15 table safely.
"""
import os
import sqlite3

import pandas as pd

REQUIRED_COLUMNS = [
    "timestamp_utc",
    "open",
    "high",
    "low",
    "close",
    "volume",
    "session",
]

def get_connection(db_path: str) -> sqlite3.Connection:
    """Open a SQLite connection and create the parent directory if necessary."""
    os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA journal_mode=WAL;")
    return conn

def _table_name(timeframe: str) -> str:
    normalized = timeframe.lower()
    if normalized not in {"m1", "m5", "m15", "h1", "h4"}:
        raise ValueError(f"Unsupported timeframe: {timeframe!r}")
    return f"ohlcv_{normalized}"

def _create_table(conn: sqlite3.Connection, table: str) -> None:
    conn.execute(f"""
        CREATE TABLE IF NOT EXISTS {table} (
            symbol TEXT NOT NULL,
            timestamp_utc INTEGER NOT NULL,
            open REAL NOT NULL,
            high REAL NOT NULL,
            low REAL NOT NULL,
            close REAL NOT NULL,
            volume REAL NOT NULL,
            session TEXT NOT NULL,
            PRIMARY KEY (symbol, timestamp_utc)
        );
    """)
    conn.execute(
        f"CREATE INDEX IF NOT EXISTS idx_{table}_symbol_ts "
        f"ON {table}(symbol, timestamp_utc);"
    )

def _has_legacy_schema(conn: sqlite3.Connection, table: str) -> bool:
    columns = conn.execute(f"PRAGMA table_info({table});").fetchall()
    if not columns:
        return False

    names = {column[1] for column in columns}
    primary_key_columns = [
        column[1]
        for column in sorted(columns, key=lambda column: column[5])
        if column[5] > 0
    ]
    return "symbol" not in names or primary_key_columns != ["symbol", "timestamp_utc"]

def init_schema(db_path: str, timeframes: list[str]) -> None:
    """

```

Create symbol-aware tables.

If a legacy single-symbol table exists, rename it once as a backup rather than silently mixing its old data with six new symbols.

```
"""
conn = get_connection(db_path)
try:
    for timeframe in timeframes:
        table = _table_name(timeframe)

        if _has_legacy_schema(conn, table):
            legacy_table = f"{table}_legacy_single_symbol"
            exists = conn.execute(
                "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?",
                (legacy_table,),
            ).fetchone()

            if exists:
                raise RuntimeError(
                    f"Legacy backup table {legacy_table!r} already exists. "
                    f"Move or delete it deliberately before migrating {table!r}."
                )

            conn.execute(f"ALTER TABLE {table} RENAME TO {legacy_table}")

        _create_table(conn, table)

    conn.commit()
finally:
    conn.close()
```

```
def upsert_candles(
    db_path: str,
    timeframe: str,
    symbol: str,
    df: pd.DataFrame,
) -> None:
    """Insert or replace one asset's candles for a timeframe."""
    missing = set(REQUIRED_COLUMNS) - set(df.columns)
    if missing:
        raise ValueError(f"DataFrame missing required columns: {sorted(missing)}")

    if df.empty:
        return

    table = _table_name(timeframe)
    init_schema(db_path, [timeframe])

    rows = [
        (
            symbol,
            int(row.timestamp_utc),
            float(row.open),
            float(row.high),
            float(row.low),
            float(row.close),
            float(row.volume),
            str(row.session),
        )
        for row in df[REQUIRED_COLUMNS].itertuples(index=False)
    ]

    conn = get_connection(db_path)
    try:
        conn.executemany(
            f"""
            INSERT INTO {table}
            (symbol, timestamp_utc, open, high, low, close, volume, session)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(symbol, timestamp_utc) DO UPDATE SET
            open=excluded.open,
            high=excluded.high,
            low=excluded.low,
            close=excluded.close,
            volume=excluded.volume,
            session=excluded.session
            """,
            rows,
        )
        conn.commit()
    finally:
        conn.close()
```

```
def read_candles(
    db_path: str,
```

```

        timeframe: str,
        symbol: str,
        start_ts: int | None = None,
        end_ts: int | None = None,
    ) -> pd.DataFrame:
        """Read one asset's candles, ascending by UTC epoch timestamp."""
        table = _table_name(timeframe)
        init_schema(db_path, [timeframe])

        query = f"SELECT * FROM {table} WHERE symbol = ?"
        params: list[object] = [symbol]

        if start_ts is not None:
            query += " AND timestamp_utc >= ?"
            params.append(int(start_ts))

        if end_ts is not None:
            query += " AND timestamp_utc <= ?"
            params.append(int(end_ts))

        query += " ORDER BY timestamp_utc ASC"

        conn = get_connection(db_path)
        try:
            return pd.read_sql_query(query, conn, params=params)
        finally:
            conn.close()

```

FILE: data\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: data\tests\test\_backfill.py

```

"""
Tests for data/ingestion.py rate-limit backoff and backfill_historical() pagination.
All HTTP calls are mocked with time.sleep patched out - no real network calls,
no real waiting.
Run with: pytest data/tests/test_backfill.py -v
"""

import os
import sys
from unittest.mock import patch, MagicMock
import pytest
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import backfill_historical, fetch_live_candles, _request_with_backoff, TWELVE_DATA_MAX_OUTPUTSIZE

CFG = load_config()
SESSIONS = CFG["sessions"]

def _make_values(start_dt: pd.Timestamp, n: int, step_minutes: int = 15):
    """Generates n descending timestamps (newest first, as Twelve Data returns)."""
    values = []
    for i in range(n):
        dt = start_dt - pd.Timedelta(minutes=step_minutes * i)
        values.append({
            "datetime": dt.strftime("%Y-%m-%d %H:%M:%S"),
            "open": "2400.0", "high": "2401.0", "low": "2399.0", "close": "2400.5", "volume": "100.0",
        })
    return values

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"})
@patch("time.sleep", return_value=None)
@patch("data.ingestion.requests.get")
def test_request_with_backoff_retries_on_429_status_code(mock_get, mock_sleep):
    resp_429 = MagicMock(status_code=429)
    resp_429.json.return_value = {}
    resp_ok = MagicMock(status_code=200)
    resp_ok.json.return_value = {"values": []}
    resp_ok.raise_for_status = MagicMock()
    mock_get.side_effect = [resp_429, resp_ok]

    result = _request_with_backoff("http://fake.url", {"apikey": "x"})
    assert result == {"values": []}
    assert mock_get.call_count == 2
    assert mock_sleep.called

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"})
@patch("time.sleep", return_value=None)
@patch("data.ingestion.requests.get")
def test_request_with_backoff_retries_on_payload_code_429(mock_get, mock_sleep):

```

```

resp_429_payload = MagicMock(status_code=200)
resp_429_payload.json.return_value = {"code": 429, "message": "rate limit"}
resp_ok = MagicMock(status_code=200)
resp_ok.json.return_value = {"values": []}
resp_ok.raise_for_status = MagicMock()
mock_get.side_effect = [resp_429_payload, resp_ok]

result = _request_with_backoff("http://fake.url", {"apikey": "x"})
assert result == {"values": []}
assert mock_get.call_count == 2

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"})
@patch("time.sleep", return_value=None)
@patch("data.ingestion.requests.get")
def test_request_with_backoff_raises_after_max_retries(mock_get, mock_sleep):
    resp_429 = MagicMock(status_code=429)
    resp_429.json.return_value = {}
    mock_get.return_value = resp_429

    with pytest.raises(RuntimeError):
        _request_with_backoff("http://fake.url", {"apikey": "x"}, max_retries=3)
    assert mock_get.call_count == 3

def test_fetch_live_candles_rejects_n_candles_over_max_outputsize():
    with patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"}):
        with pytest.raises(ValueError, match="exceeds Twelve Data"):
            fetch_live_candles("M15", TWELVE_DATA_MAX_OUTPUTSIZE + 1, SESSIONS)

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"})
@patch("time.sleep", return_value=None)
@patch("data.ingestion.requests.get")
def test_backfill_historical_paginates_across_multiple_calls(mock_get, mock_sleep):
    """
    Simulate a range requiring 2 pages: first call returns 5000 rows ending at some
    cursor, second call returns the remaining rows and then empty to stop.
    """
    end_dt = pd.Timestamp("2026-01-10 00:00:00", tz="UTC")
    page1_values = _make_values(end_dt, 100, step_minutes=15) # newest first
    page2_values = _make_values(page1_values and pd.Timestamp(page1_values[-1]["datetime"], tz="UTC") - pd.Timedelta(minutes=15), 50,
    step_minutes=15)

    resp1 = MagicMock(status_code=200)
    resp1.json.return_value = {"values": page1_values}
    resp1.raise_for_status = MagicMock()

    resp2 = MagicMock(status_code=200)
    resp2.json.return_value = {"values": page2_values}
    resp2.raise_for_status = MagicMock()

    resp_empty = MagicMock(status_code=200)
    resp_empty.json.return_value = {"values": []}
    resp_empty.raise_for_status = MagicMock()

    mock_get.side_effect = [resp1, resp2, resp_empty]

    start_date = (end_dt - pd.Timedelta(days=5)).strftime("%Y-%m-%d")
    end_date = end_dt.strftime("%Y-%m-%d")

    df = backfill_historical("M15", start_date, end_date, SESSIONS)
    assert len(df) > 0
    assert df["timestamp_utc"].is_monotonic_increasing
    assert not df["timestamp_utc"].duplicated().any()
    assert mock_get.call_count >= 2

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"})
@patch("time.sleep", return_value=None)
@patch("data.ingestion.requests.get")
def test_backfill_historical_raises_on_zero_rows(mock_get, mock_sleep):
    resp_empty = MagicMock(status_code=200)
    resp_empty.json.return_value = {"values": []}
    resp_empty.raise_for_status = MagicMock()
    mock_get.return_value = resp_empty

    with pytest.raises(RuntimeError):
        backfill_historical("M15", "2026-01-01", "2026-01-05", SESSIONS)

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"})
@patch("time.sleep", return_value=None)
@patch("data.ingestion.requests.get")
def test_backfill_historical_deduplicates_overlapping_timestamps(mock_get, mock_sleep):
    end_dt = pd.Timestamp("2026-01-10 00:00:00", tz="UTC")
    overlapping_values = _make_values(end_dt, 20, step_minutes=15)

```

```

respl = MagicMock(status_code=200)
respl.json.return_value = {"values": overlapping_values}
respl.raise_for_status = MagicMock()

resp_empty = MagicMock(status_code=200)
resp_empty.json.return_value = {"values": []}
resp_empty.raise_for_status = MagicMock()

mock_get.side_effect = [respl, resp_empty]

start_date = (end_dt - pd.Timedelta(days=1)).strftime("%Y-%m-%d")
df = backfill_historical("M15", start_date, end_dt.strftime("%Y-%m-%d"), SESSIONS)
assert not df["timestamp_utc"].duplicated().any()

```

FILE: data\tests\test\_ingestion.py

```

"""
Unit tests for data/ ingestion, storage, and session tagging.
Run with: pytest data/tests/test_ingestion.py -v
"""
import os
import sys
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import fetch_mock_candles, TIMEFRAME_TO_SECONDS
from data.storage import init_schema, upsert_candles, read_candles
from data.session_tagger import tag_session

CFG = load_config()
SESSIONS = CFG["sessions"]

def test_mock_candles_shape():
    df = fetch_mock_candles("M5", n_candles=100, sessions_config=SESSIONS)
    assert len(df) == 100
    assert set(["timestamp_utc", "open", "high", "low", "close", "volume", "session"]) <= set(df.columns)

def test_mock_candles_timestamp_monotonic():
    df = fetch_mock_candles("M15", n_candles=50, sessions_config=SESSIONS)
    diffs = df["timestamp_utc"].diff().dropna()
    assert (diffs == TIMEFRAME_TO_SECONDS["M15"]).all(), "Timestamps must be evenly spaced with no gaps"

def test_mock_candles_ohlc_consistency():
    df = fetch_mock_candles("M1", n_candles=200, sessions_config=SESSIONS)
    assert (df["high"] >= df["open"]).all()
    assert (df["high"] >= df["close"]).all()
    assert (df["low"] <= df["open"]).all()
    assert (df["low"] <= df["close"]).all()

def test_session_tagging_known_hours():
    # 03:00 UTC -> Asia only window (0-8)
    ts_asia = pd.Timestamp("2026-07-24 03:00:00", tz="UTC").timestamp()
    label = tag_session(ts_asia, SESSIONS)
    assert "asia" in label

    # 13:00 UTC -> London (7-16) and NY (12-21) overlap
    ts_overlap = pd.Timestamp("2026-07-24 13:00:00", tz="UTC").timestamp()
    label_overlap = tag_session(ts_overlap, SESSIONS)
    assert "london" in label_overlap and "ny" in label_overlap

    # 23:00 UTC -> off_session (outside all windows)
    ts_off = pd.Timestamp("2026-07-24 23:00:00", tz="UTC").timestamp()
    label_off = tag_session(ts_off, SESSIONS)
    assert label_off == "off_session"

def test_storage_roundtrip(tmp_path):
    db_path = str(tmp_path / "test.sqlite")
    init_schema(db_path, ["M5"])
    df = fetch_mock_candles("M5", n_candles=30, sessions_config=SESSIONS)
    upsert_candles(db_path, "M5", df)
    read_back = read_candles(db_path, "M5")
    assert len(read_back) == 30
    assert list(read_back["timestamp_utc"]) == sorted(read_back["timestamp_utc"])

def test_storage_idempotent_upsert(tmp_path):
    """Re-inserting the same candles must not create duplicates (idempotency check)."""
    db_path = str(tmp_path / "test2.sqlite")
    init_schema(db_path, ["M1"])

```

```

df = fetch_mock_candles("M1", n_candles=20, sessions_config=SESSIONS)
upsert_candles(db_path, "M1", df)
upsert_candles(db_path, "M1", df) # insert same rows again
read_back = read_candles(db_path, "M1")
assert len(read_back) == 20 # no duplicates due to INSERT OR REPLACE on PK

```

FILE: data\tests\test\_ingestion\_live.py

```

"""
Tests for data/ingestion.py::fetch_live_candles - HTTP calls are fully mocked,
no real network requests are made. Validates response parsing, error handling,
and the unified fetch_candles() dispatcher.
"""

import os
import sys
import time
from unittest.mock import patch, MagicMock
import pytest
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import fetch_live_candles, fetch_candles

CFG = load_config()
SESSIONS = CFG["sessions"]

def _mock_twelvedata_payload(n=5):
    values = []
    base_ts = 1700000000
    for i in range(n):
        ts = base_ts + i * 900
        dt = pd.Timestamp(ts, unit="s", tz="UTC").strftime("%Y-%m-%d %H:%M:%S")
        values.append({
            "datetime": dt, "open": "2400.5", "high": "2402.0",
            "low": "2399.0", "close": "2401.0", "volume": "150.0",
        })
    return {"values": values}

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key_for_testing"})
@patch("data.ingestion.requests.get")
def test_fetch_live_candles_parses_valid_response(mock_get):
    mock_resp = MagicMock()
    mock_resp.json.return_value = _mock_twelvedata_payload(5)
    mock_resp.raise_for_status = MagicMock()
    mock_get.return_value = mock_resp

    df = fetch_live_candles("M15", 5, SESSIONS)
    assert len(df) == 5
    assert list(df.columns[:6]) == ["timestamp_utc", "open", "high", "low", "close", "volume"]
    assert df["timestamp_utc"].is_monotonic_increasing
    assert "session" in df.columns
    assert mock_get.called

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key_for_testing"})
@patch("data.ingestion.requests.get")
def test_fetch_live_candles_raises_on_malformed_response(mock_get):
    mock_resp = MagicMock()
    mock_resp.json.return_value = {"error": "invalid symbol"}
    mock_resp.raise_for_status = MagicMock()
    mock_get.return_value = mock_resp

    with pytest.raises(RuntimeError):
        fetch_live_candles("M15", 5, SESSIONS)

def test_fetch_live_candles_raises_without_api_key():
    """Must fail loudly if API key env var is missing - never silently fall back to mock."""
    env_backup = os.environ.pop("TWELVE_DATA_API_KEY", None)
    try:
        with pytest.raises(EnvironmentError):
            fetch_live_candles("M15", 5, SESSIONS)
    finally:
        if env_backup is not None:
            os.environ["TWELVE_DATA_API_KEY"] = env_backup

def test_fetch_live_candles_rejects_unsupported_timeframe():
    with patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key"}):
        with pytest.raises(ValueError):
            fetch_live_candles("M30", 5, SESSIONS)

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key_for_testing"})

```

```

@patch("data.ingestion.requests.get")
def test_fetch_candles_dispatcher_routes_to_live(mock_get):
    mock_resp = MagicMock()
    mock_resp.json.return_value = _mock_twelvedata_payload(3)
    mock_resp.raise_for_status = MagicMock()
    mock_get.return_value = mock_resp

    df = fetch_candles("M15", 3, SESSIONS, mode="live")
    assert len(df) == 3
    assert mock_get.called

def test_fetch_candles_dispatcher_routes_to_mock():
    df = fetch_candles("M15", 10, SESSIONS, mode="mock")
    assert len(df) == 10

def test_fetch_candles_dispatcher_rejects_unknown_mode():
    with pytest.raises(ValueError):
        fetch_candles("M15", 10, SESSIONS, mode="bogus")

@patch.dict(os.environ, {"TWELVE_DATA_API_KEY": "fake_key_for_testing"})
@patch("data.ingestion.requests.get")
def test_fetch_live_candles_raises_on_http_error(mock_get):
    mock_resp = MagicMock()
    mock_resp.raise_for_status.side_effect = Exception("503 Service Unavailable")
    mock_get.return_value = mock_resp

    with pytest.raises(Exception):
        fetch_live_candles("M15", 5, SESSIONS)

```

FILE: data\tests\test\_signal\_log.py

```

"""
Unit tests for data/signal_log.py.
Run with: pytest data/tests/test_signal_log.py -v
"""
import os
import sys
import json

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from data.signal_log import init_schema, log_signal, read_signal_history

def _sample_signal(ts=1700000000, bias="long"):
    return {
        "bias": bias,
        "confidence": 0.85,
        "entry_zone": [2400.0, 2400.5] if bias != "no_trade" else None,
        "invalidation": 2398.0 if bias != "no_trade" else None,
        "targets": [2403.0] if bias != "no_trade" else None,
        "reasoning_summary": "test reasoning",
        "regime": "trend_up",
        "timestamp_utc": ts,
        "session": "london",
        "generated_at": "2026-07-24T08:00:00+00:00",
    }

def test_init_schema_creates_table(tmp_path):
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)
    assert os.path.exists(db_path)

def test_log_and_read_signal_roundtrip(tmp_path):
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)
    signal = _sample_signal()
    log_signal(db_path, signal, alert_sent=True)

    df = read_signal_history(db_path)
    assert len(df) == 1
    assert df.iloc[0]["bias"] == "long"
    assert df.iloc[0]["alert_sent"] == 1
    assert json.loads(df.iloc[0]["entry_zone"]) == [2400.0, 2400.5]

def test_log_signal_idempotent_on_same_timestamp(tmp_path):
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)
    signal = _sample_signal(ts=1700000000)
    log_signal(db_path, signal, alert_sent=False)
    signal_updated = _sample_signal(ts=1700000000, bias="short")
    log_signal(db_path, signal_updated, alert_sent=True)

```



```

df = read_signal_history(db_path)
assert len(df) == 1, "Same timestamp must upsert, not duplicate"
assert df.iloc[0]["bias"] == "short"
assert df.iloc[0]["alert_sent"] == 1

def test_log_no_trade_signal_has_null_trade_fields(tmp_path):
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)
    signal = _sample_signal(ts=1700000900, bias="no_trade")
    log_signal(db_path, signal, alert_sent=False)

    df = read_signal_history(db_path)
    assert df.iloc[0]["entry_zone"] is None
    assert df.iloc[0]["targets"] is None

def test_read_signal_history_filters_by_time_range(tmp_path):
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)
    for i in range(5):
        log_signal(db_path, _sample_signal(ts=1700000000 + i * 900), alert_sent=False)

    df = read_signal_history(db_path, start_ts=1700000900, end_ts=1700002700)
    assert len(df) == 3
    assert df["timestamp_utc"].min() >= 1700000900
    assert df["timestamp_utc"].max() <= 1700002700

def test_read_signal_history_sorted_ascending(tmp_path):
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)
    for ts in [1700002700, 1700000000, 1700001800]:
        log_signal(db_path, _sample_signal(ts=ts), alert_sent=False)

    df = read_signal_history(db_path)
    assert list(df["timestamp_utc"]) == sorted(df["timestamp_utc"])

```

FILE: features\\_\_init\_\_.py

# [EMPTY FILE]

FILE: features\candle\_anatomy.py

```

"""
Candle anatomy features: body ratio, upper/lower wick ratios, candle direction.
All computed row-wise from that row's own OHLC only - inherently causal, zero look-ahead risk.
"""

import numpy as np
import pandas as pd

def candle_anatomy(df: pd.DataFrame) -> pd.DataFrame:
    """
    body_ratio: |close-open| / (high-low) -> how much of the candle range is "body"
    upper_wick_ratio: (high - max(open,close)) / (high-low)
    lower_wick_ratio: (min(open,close) - low) / (high-low)
    direction: +1 bullish, -1 bearish, 0 doji (open == close)
    All ratios are NaN-safe when high == low (zero-range candle, rare but possible on illiquid feeds).
    """
    out = df.copy()
    rng = (out["high"] - out["low"]).replace(0, np.nan)

    body = (out["close"] - out["open"]).abs()
    upper_wick = out["high"] - out[["open", "close"]].max(axis=1)
    lower_wick = out[["open", "close"]].min(axis=1) - out["low"]

    out["body_ratio"] = (body / rng).fillna(0.0)
    out["upper_wick_ratio"] = (upper_wick / rng).fillna(0.0)
    out["lower_wick_ratio"] = (lower_wick / rng).fillna(0.0)
    out["candle_direction"] = np.sign(out["close"] - out["open"]).astype(int)

    return out

```

FILE: features\indicators.py

```

"""
Technical indicators: EMA, RSI, MACD, ATR, Bollinger Bands width.

CRITICAL NO-LOOK-AHEAD RULE:
Every function here computes indicator value at row i using ONLY rows [0..i].
This is naturally satisfied by pandas rolling/ewm operations because they are
causal by construction (a rolling window ending at i never touches i+1..n).
We still add explicit unit tests (see features/tests/test_no_lookahead.py) that
truncate the DataFrame at index i and assert the value is identical to the value
computed on the full DataFrame at row i - this proves no future leakage empirically,
not just by code inspection.

```

```

"""
import numpy as np
import pandas as pd

def ema(series: pd.Series, period: int) -> pd.Series:
    """Exponential moving average. adjust=False = standard trading EMA recursion, causal."""
    return series.ewm(span=period, adjust=False).mean()

def rsi(series: pd.Series, period: int = 14) -> pd.Series:
    """Relative Strength Index (Wilder's smoothing via ewm, causal)."""
    delta = series.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain = gain.ewm(alpha=1 / period, adjust=False, min_periods=period).mean()
    avg_loss = loss.ewm(alpha=1 / period, adjust=False, min_periods=period).mean()
    rs = avg_gain / avg_loss.replace(0, np.nan)
    out = 100 - (100 / (1 + rs))
    return out.fillna(50.0) # neutral RSI during warm-up, not NaN, to keep downstream code simple

def macd(series: pd.Series, fast: int = 12, slow: int = 26, signal: int = 9) -> pd.DataFrame:
    """MACD line, signal line, and histogram. All causal (ewm-based)."""
    ema_fast = ema(series, fast)
    ema_slow = ema(series, slow)
    macd_line = ema_fast - ema_slow
    signal_line = ema(macd_line, signal)
    hist = macd_line - signal_line
    return pd.DataFrame({"macd_line": macd_line, "macd_signal": signal_line, "macd_hist": hist})

def atr(df: pd.DataFrame, period: int = 14) -> pd.Series:
    """
    Average True Range. True range uses previous close (df['close'].shift(1)),
    which is intentional and correct (TR is defined using prior close) - this is
    NOT look-ahead because shift(1) pulls from the PAST, not the future.
    """
    prev_close = df["close"].shift(1)
    tr1 = df["high"] - df["low"]
    tr2 = (df["high"] - prev_close).abs()
    tr3 = (df["low"] - prev_close).abs()
    true_range = pd.concat([tr1, tr2, tr3], axis=1).max(axis=1)
    return true_range.ewm(alpha=1 / period, adjust=False, min_periods=period).mean()

def bollinger_width(series: pd.Series, period: int = 20, std_dev: float = 2.0) -> pd.DataFrame:
    """Bollinger Bands and normalized width (width / middle band, causal rolling stats)."""
    mid = series.rolling(window=period, min_periods=period).mean()
    std = series.rolling(window=period, min_periods=period).std()
    upper = mid + std_dev * std
    lower = mid - std_dev * std
    width = (upper - lower) / mid.replace(0, np.nan)
    return pd.DataFrame({"bb_mid": mid, "bb_upper": upper, "bb_lower": lower, "bb_width": width})

def build_all_indicators(df: pd.DataFrame, cfg: dict) -> pd.DataFrame:
    """
    Compute the full indicator set from config-driven parameters (no hardcoded periods here).
    df must contain columns: open, high, low, close, volume, timestamp_utc.
    Returns a new DataFrame with indicator columns appended, same row count and index as input.
    """
    out = df.copy()
    for period in cfg["features"]["ema_periods"]:
        out[f"ema_{period}"] = ema(out["close"], period)

    out["rsi"] = rsi(out["close"], cfg["features"]["rsi_period"])

    macd_cfg = cfg["features"]["macd"]
    macd_df = macd(out["close"], macd_cfg["fast"], macd_cfg["slow"], macd_cfg["signal"])
    out = pd.concat([out, macd_df], axis=1)

    out["atr"] = atr(out, cfg["features"]["atr_period"])

    bb_cfg = cfg["features"]["bollinger"]
    bb_df = bollinger_width(out["close"], bb_cfg["period"], bb_cfg["std_dev"])
    out = pd.concat([out, bb_df], axis=1)

    return out

```

FILE: features\mtf\_confluence.py

```

"""
Multi-timeframe confluence score.

Combines directional bias signals from higher timeframes (e.g. M15, H1, H4) into
a single confluence score for a base timeframe (e.g. M5) row.

```

CRITICAL NO-LOOK-AHEAD NOTE:

When merging a higher-timeframe (HTF) feature onto a lower-timeframe (LTF) row, we must only use the HTF candle that had ALREADY CLOSED at or before the LTF candle's timestamp. merge\_asof with direction="backward" enforces exactly this: it matches each LTF timestamp to the most recent HTF timestamp <= it, never a future HTF candle. This is the single most common source of look-ahead bias in multi-timeframe systems, so it is isolated and heavily commented here.

"""

import pandas as pd

```
def merge_htf_feature(ltf_df: pd.DataFrame, htf_df: pd.DataFrame, feature_col: str,
                     out_col_name: str) -> pd.DataFrame:
    """
    Backward-merge a single HTF feature column onto the LTF DataFrame using merge_asof.
    Both DataFrames must be sorted ascending by timestamp_utc (enforced here defensively).
    """
    ltf_sorted = ltf_df.sort_values("timestamp_utc").reset_index(drop=True)
    htf_sorted = htf_df[["timestamp_utc", feature_col]].sort_values("timestamp_utc").reset_index(drop=True)
    htf_sorted = htf_sorted.rename(columns={feature_col: out_col_name})

    merged = pd.merge_asof(
        ltf_sorted, htf_sorted,
        on="timestamp_utc",
        direction="backward", # NEVER "forward" or "nearest" - would leak future HTF candles
    )
    return merged
```

```
def compute_confluence_score(ltf_df: pd.DataFrame, htf_frames: dict, cfg: dict) -> pd.DataFrame:
    """
    htf_frames: dict of {timeframe_name: htf_dataframe_with_indicators}
    For each HTF, derive a simple directional vote: +1 if close > ema_50, -1 if close < ema_50, else 0.
    Confluence score = sum of votes across configured mtf_reference_timeframes, normalized to [-1, 1].
    """
    out = ltf_df.copy()
    votes = []
    ref_tfs = cfg["features"]["mtf_reference_timeframes"]

    for tf_name in ref_tfs:
        htf_df = htf_frames.get(tf_name)
        if htf_df is None:
            continue
        htf_df = htf_df.copy()
        htf_df["htf_vote"] = 0
        htf_df.loc[htf_df["close"] > htf_df["ema_50"], "htf_vote"] = 1
        htf_df.loc[htf_df["close"] < htf_df["ema_50"], "htf_vote"] = -1

        out = merge_htf_feature(out, htf_df, "htf_vote", f"vote_{tf_name}")
        votes.append(f"vote_{tf_name}")

    if votes:
        out["mtf_confluence_score"] = out[votes].sum(axis=1) / len(votes)
    else:
        out["mtf_confluence_score"] = 0.0

    return out
```

FILE: features/structure.py

"""

Structure highs/lows detection (swing points).

CRITICAL NO-LOOK-AHEAD NOTE:

A "true" swing high at index i is normally defined by comparing candle i to K candles on BOTH sides (i-K..i+K). That definition is look-ahead by nature - you cannot confirm a swing high until K candles AFTER it close. To keep this causal for real-time use, we implement a CONFIRMED swing detector: the swing at index (i - lookback) is only marked/confirmed once we have processed candle i, i.e. the swing label appears with a K-candle delay. This delay is intentional and documented - callers must be aware a swing flagged at row j means "a swing was confirmed AS OF this row, referring to a candle 'lookback' rows ago". We never let row i's OWN swing\_high/swing\_low flag depend on rows AFTER i.

"""

import numpy as np  
import pandas as pd

```
def detect_structure(df: pd.DataFrame, lookback: int = 20) -> pd.DataFrame:
    """
    Adds columns:
        swing_high_confirmed: True at row i if the candle at (i - lookback) is a
                               confirmed local high relative to the lookback-window on ITS left
                               and the lookback rows up to and including row i on its right.
        swing_low_confirmed: symmetric for lows.
        last_structure_high / last_structure_low: forward-filled most recent confirmed level,
                                                    known at row i using only data up to row i (causal).
    """
```

```

out = df.copy()
n = len(out)
swing_high = np.zeros(n, dtype=bool)
swing_low = np.zeros(n, dtype=bool)

highs = out["high"].values
lows = out["low"].values

# Confirm swing at candidate index c only once we have `lookback` candles AFTER c (i.e. current index i = c + lookback)
for i in range(lookback * 2, n):
    c = i - lookback # candidate swing index, now fully bracketed by known past+future-up-to-i data
    window_left = highs[c - lookback:c]
    window_right = highs[c + 1:c + lookback + 1]
    if len(window_left) == lookback and len(window_right) == lookback:
        if highs[c] > window_left.max() and highs[c] > window_right.max():
            swing_high[i] = True # flag appears at i, referring to candidate c

    window_left_l = lows[c - lookback:c]
    window_right_l = lows[c + 1:c + lookback + 1]
    if len(window_left_l) == lookback and len(window_right_l) == lookback:
        if lows[c] < window_left_l.min() and lows[c] < window_right_l.min():
            swing_low[i] = True

out["swing_high_confirmed"] = swing_high
out["swing_low_confirmed"] = swing_low

# Track the level value at the confirmed candidate index, forward-filled causally
confirmed_high_level = pd.Series(np.where(swing_high, highs[np.maximum(np.arange(n) - lookback, 0)], np.nan), index=out.index)
confirmed_low_level = pd.Series(np.where(swing_low, lows[np.maximum(np.arange(n) - lookback, 0)], np.nan), index=out.index)

out["last_structure_high"] = confirmed_high_level.ffill()
out["last_structure_low"] = confirmed_low_level.ffill()

return out

```

FILE: features\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: features\tests\test\_no\_lookahead.py

```

"""
No-look-ahead proof tests for features/.

Core methodology: for each feature function, compute the feature on the FULL
dataset, then recompute it on a TRUNCATED dataset (rows [0..i] only). If the
value at row i differs between the two computations, the feature is using
future data at row i - this is exactly the bug class we must prevent.

Run with: pytest features/tests/test_no_lookahead.py -v
"""
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import fetch_mock_candles
from features.indicators import ema, rsi, macd, atr, bollinger_width, build_all_indicators
from features.candle_anatomy import candle_anatomy
from features.structure import detect_structure
from features.mtf_confluence import merge_hmf_feature

CFG = load_config()
SESSIONS = CFG["sessions"]

def _sample_df(n=300):
    return fetch_mock_candles("M5", n_candles=n, sessions_config=SESSIONS)

def test_ema_no_lookahead():
    df = _sample_df()
    i = 150
    full = ema(df["close"], 21)
    truncated = ema(df["close"].iloc[:i + 1], 21)
    assert np.isclose(full.iloc[i], truncated.iloc[i], rtol=1e-9)

def test_rsi_no_lookahead():
    df = _sample_df()
    i = 150
    full = rsi(df["close"], 14)
    truncated = rsi(df["close"].iloc[:i + 1], 14)
    assert np.isclose(full.iloc[i], truncated.iloc[i], rtol=1e-6)

```

```

def test_macd_no_lookahead():
    df = _sample_df()
    i = 150
    full = macd(df["close"], 12, 26, 9)
    truncated = macd(df["close"].iloc[:i + 1], 12, 26, 9)
    assert np.isclose(full["macd_line"].iloc[i], truncated["macd_line"].iloc[i], rtol=1e-9)

def test_atr_no_lookahead():
    df = _sample_df()
    i = 150
    full = atr(df, 14)
    truncated = atr(df.iloc[:i + 1], 14)
    assert np.isclose(full.iloc[i], truncated.iloc[i], rtol=1e-6)

def test_bollinger_no_lookahead():
    df = _sample_df()
    i = 150
    full = bollinger_width(df["close"], 20, 2.0)
    truncated = bollinger_width(df["close"].iloc[:i + 1], 20, 2.0)
    assert np.isclose(full["bb_width"].iloc[i], truncated["bb_width"].iloc[i], rtol=1e-6)

def test_build_all_indicators_no_lookahead():
    """Composite test across the full indicator pipeline at once."""
    df = _sample_df(n=400)
    i = 250
    full = build_all_indicators(df, CFG)
    truncated = build_all_indicators(df.iloc[:i + 1].copy(), CFG)
    check_cols = ["ema_9", "ema_21", "rsi", "macd_line", "atr", "bb_width"]
    for col in check_cols:
        assert np.isclose(full[col].iloc[i], truncated[col].iloc[i], rtol=1e-6, equal_nan=True), f"Leak in {col}"

def test_candle_anatomy_no_lookahead():
    """Row-wise features must be identical regardless of dataset length."""
    df = _sample_df()
    i = 150
    full = candle_anatomy(df)
    truncated = candle_anatomy(df.iloc[:i + 1].copy())
    assert np.isclose(full["body_ratio"].iloc[i], truncated["body_ratio"].iloc[i])
    assert full["candle_direction"].iloc[i] == truncated["candle_direction"].iloc[i]

def test_structure_swing_detection_no_lookahead():
    """
    A swing flag confirmed at row i using the full dataset must ALSO be confirmed
    at row i when only rows [0..i] are available - because by construction the
    confirmation only ever looks lookback candles into the past relative to i.
    """
    df = _sample_df(n=300)
    lookback = 20
    i = 200
    full = detect_structure(df, lookback=lookback)
    truncated = detect_structure(df.iloc[:i + 1].copy(), lookback=lookback)
    assert full["swing_high_confirmed"].iloc[i] == truncated["swing_high_confirmed"].iloc[i]
    assert full["swing_low_confirmed"].iloc[i] == truncated["swing_low_confirmed"].iloc[i]
    assert full["last_structure_high"].iloc[i] == truncated["last_structure_high"].iloc[i] or (
        pd.isna(full["last_structure_high"].iloc[i]) and pd.isna(truncated["last_structure_high"].iloc[i])
    )

def test_mtf_merge_asof_never_uses_future_hft_candle():
    """
    Explicit test that merge_asof backward direction never pulls an HFT candle
    whose timestamp is AFTER the LTF row's timestamp - the classic MTF leak.
    """
    ltf_df = _sample_df(n=100) # M5
    htf_df = fetch_mock_candles("H1", n_candles=20, sessions_config=SESSIONS,
                                end_ts=int(ltf_df["timestamp_utc"].max()))
    htf_df["dummy_feature"] = htf_df["close"]

    merged = merge_hft_feature(ltf_df, htf_df, "dummy_feature", "hft_dummy")

    htf_sorted = htf_df.sort_values("timestamp_utc").reset_index(drop=True)
    for idx, row in merged.iterrows():
        if pd.isna(row["hft_dummy"]):
            continue
        matched_hft_ts = htf_sorted.loc[htf_sorted["close"] == row["hft_dummy"], "timestamp_utc"]
        # The matched HFT candle's timestamp must always be <= the LTF row's timestamp
        valid_ts = htf_sorted[htf_sorted["timestamp_utc"] <= row["timestamp_utc"]]
        if len(valid_ts) > 0:
            expected_ts = valid_ts["timestamp_utc"].max()
            actual_hft_row = htf_sorted[htf_sorted["close"] == row["hft_dummy"]]
            if len(actual_hft_row) > 0:

```

```
assert actual_htf_row["timestamp_utc"].iloc[0] <= row["timestamp_utc"], \
    "Look-ahead detected: merge_asof pulled a future HTF candle!"
```

FILE: fix\_ingestion.py

```
import re

with open('data/ingestion.py', 'r') as f:
    content = f.read()

new_func = (
    'def backfill_historical(timeframe: str, start_date: str, end_date: str, sessions_config: dict,\n'
    '    '
    '        api_key_env: str = "TWELVE_DATA_API_KEY",\n'
    '        base_url: str = "https://api.twelvedata.com/time_series") -> pd.DataFrame:\n'
    '    '
    '    """Pull historical candles in 500-row chunks (free-tier safe).""" \n'
    '    api_key = get_env(api_key_env, required=True)\n'
    '    interval_map = {"M1": "1min", "M5": "5min", "M15": "15min", "H1": "1h", "H4": "4h"}\n'
    '    if timeframe not in interval_map:\n'
    '        raise ValueError(f"Unsupported timeframe: {timeframe}")\n'
    '    CHUNK_SIZE = 500\n'
    '    all_rows = []\n'
    '    current_end = end_date\n'
    '    while True:\n'
    '        params = {\n'
    '            "symbol": "XAU/USD",\n'
    '            "interval": interval_map[timeframe],\n'
    '            "outputsize": CHUNK_SIZE,\n'
    '            "end_date": current_end,\n'
    '            "apikey": api_key,\n'
    '            "timezone": "UTC",\n'
    '            "format": "JSON",\n'
    '        }\n'
    '        data = _request_with_backoff(base_url, params)\n'
    '        values = data.get("values", [])\n'
    '        if not values:\n'
    '            break\n'
    '        values = [v for v in values if v["datetime"][:10] >= start_date]\n'
    '        if not values:\n'
    '            break\n'
    '        all_rows.extend(values)\n'
    '        print(f"    {len(all_rows)} candles fetched", end="\r", flush=True)\n'
    '        oldest_ts = values[-1]["datetime"]\n'
    '        if oldest_ts[:10] <= start_date:\n'
    '            break\n'
    '        current_end = oldest_ts\n'
    '        import time as _t; _t.sleep(8)\n'
    '    print()\n'
    '    if not all_rows:\n'
    '        raise RuntimeError(f"No data returned for {timeframe} between {start_date} and {end_date}")\n'
    '    df = pd.DataFrame(all_rows)\n'
    '    df["datetime"] = pd.to_datetime(df["datetime"])\n'
    '    df = df.drop_duplicates(subset="datetime").sort_values("datetime").reset_index(drop=True)\n'
    '    for col in ["open", "high", "low", "close"]:\n'
    '        df[col] = df[col].astype(float)\n'
    '    df["volume"] = pd.to_numeric(df.get("volume", 0.0), errors="coerce").fillna(0.0)\n'
    '    df["timestamp_utc"] = df["datetime"].astype("int64") // 10**9\n'
    '    df = df.drop(columns=["datetime"])\n'
    '    df = df.drop_duplicates(subset="timestamp_utc").sort_values("timestamp_utc").reset_index(drop=True)\n'
    '    df = tag_dataframe(df, sessions_config)\n'
    '    return df'
)

content = re.sub(r'def backfill_historical.*?return df', new_func, content, flags=re.DOTALL)

with open('data/ingestion.py', 'w') as f:
    f.write(content)
print('Done')
```

FILE: labeling\\_\_init\_\_.py

```
# [EMPTY FILE]
```

FILE: labeling\label\_generator.py

```
"""
Label generator for supervised learning.

Supports:
- fixed barriers: absolute target/stop distances
- atr_scaled barriers: target/stop distances derived from the row's ATR

CRITICAL NO-LOOK-AHEAD WARNING:
This module is intentionally forward-looking for OFFLINE labeling only.
"""
import numpy as np
import pandas as pd
```

```

def generate_labels(df: pd.DataFrame, target_x: float, stop_y: float, horizon_n: int,
                  price_col: str = "close") -> pd.Series:
    n = len(df)
    highs = df["high"].values
    lows = df["low"].values
    entry_prices = df[price_col].values

    labels = np.full(n, np.nan)

    for i in range(n):
        if i + horizon_n >= n:
            continue

        entry = entry_prices[i]
        upper_barrier = entry + target_x
        lower_barrier = entry - stop_y

        outcome = 0
        for j in range(i + 1, i + horizon_n + 1):
            hit_upper = highs[j] >= upper_barrier
            hit_lower = lows[j] <= lower_barrier
            if hit_upper and hit_lower:
                outcome = -1
                break
            elif hit_upper:
                outcome = 1
                break
            elif hit_lower:
                outcome = -1
                break

        labels[i] = outcome

    return pd.Series(labels, index=df.index, name="label")

def generate_labels_atr_scaled(
    df: pd.DataFrame,
    target_atr_multiplier: float,
    stop_atr_multiplier: float,
    horizon_n: int,
    price_col: str = "close",
    atr_col: str = "atr",
) -> pd.Series:
    """
    Triple-barrier labels with per-row barrier widths scaled by ATR.
    Rows with missing/nonpositive ATR are left as NaN.
    """
    n = len(df)
    highs = df["high"].values
    lows = df["low"].values
    entry_prices = df[price_col].values
    atr_values = df[atr_col].values

    labels = np.full(n, np.nan)

    for i in range(n):
        if i + horizon_n >= n:
            continue

        atr_i = atr_values[i]
        if pd.isna(atr_i) or atr_i <= 0:
            continue

        entry = entry_prices[i]
        upper_barrier = entry + atr_i * target_atr_multiplier
        lower_barrier = entry - atr_i * stop_atr_multiplier

        outcome = 0
        for j in range(i + 1, i + horizon_n + 1):
            hit_upper = highs[j] >= upper_barrier
            hit_lower = lows[j] <= lower_barrier
            if hit_upper and hit_lower:
                outcome = -1
                break
            elif hit_upper:
                outcome = 1
                break
            elif hit_lower:
                outcome = -1
                break

        labels[i] = outcome

    return pd.Series(labels, index=df.index, name="label")

```

```

def generate_labels_from_config(df: pd.DataFrame, cfg: dict) -> pd.Series:
    lab_cfg = cfg["labeling"]
    method = lab_cfg.get("method", "fixed")

    if method == "fixed":
        return generate_labels(
            df,
            target_x=lab_cfg["target_pips_x"],
            stop_y=lab_cfg["stop_pips_y"],
            horizon_n=lab_cfg["horizon_candles_n"],
        )

    if method == "atr_scaled":
        return generate_labels_atr_scaled(
            df,
            target_atr_multiplier=lab_cfg["target_atr_multiplier"],
            stop_atr_multiplier=lab_cfg["stop_atr_multiplier"],
            horizon_n=lab_cfg["horizon_candles_n"],
            atr_col=lab_cfg.get("atr_column", "atr"),
        )

    raise ValueError(f"Unknown labeling.method: {method}")

def label_distribution_summary(labels: pd.Series) -> dict:
    valid = labels.dropna()
    total = len(valid)
    if total == 0:
        return {"total_valid": 0}
    return {
        "total_valid": total,
        "pct_upper_hit": float((valid == 1).sum() / total * 100),
        "pct_lower_hit": float((valid == -1).sum() / total * 100),
        "pct_no_hit": float((valid == 0).sum() / total * 100),
        "nan_count": int(labels.isna().sum()),
    }

```

FILE: labeling\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: labeling\tests\test\_labels.py

```

"""
Unit tests for labeling/label_generator.py.
Validates label correctness on constructed synthetic price paths where the
correct outcome is known exactly, plus statistical sanity checks on distribution.
Run with: pytest labeling/tests/test_labels.py -v
"""
import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import fetch_mock_candles
from labeling.label_generator import generate_labels, generate_labels_from_config, label_distribution_summary

CFG = load_config()
SESSIONS = CFG["sessions"]

# Override labeling barriers to match mock data scale (~0.15 pts/candle drift)
import copy
CFG = copy.deepcopy(CFG)
CFG["labeling"]["target_pips_x"] = 3.0
CFG["labeling"]["stop_pips_y"] = 2.0

def _flat_df(n, price=2000.0):
    return pd.DataFrame({
        "timestamp_utc": np.arange(n) * 900,
        "open": np.full(n, price),
        "high": np.full(n, price),
        "low": np.full(n, price),
        "close": np.full(n, price),
        "volume": np.full(n, 100.0),
    })

def test_upper_barrier_hit_produces_label_1():
    """Construct a path where price rises past +X well before -Y or N expires."""
    n = 20
    df = _flat_df(n)
    # At candle 5, price spikes up past the upper barrier
    df.loc[5, "high"] = 2000.0 + 200.0 # target_x default in config is 150
    labels = generate_labels(df, target_x=150, stop_y=100, horizon_n=10)

```



```

assert labels.iloc[0] == 1

def test_lower_barrier_hit_produces_label_minus_1():
    """Construct a path where price falls past -Y before hitting +X."""
    n = 20
    df = _flat_df(n)
    df.loc[3, "low"] = 2000.0 - 120.0 # stop_y = 100
    labels = generate_labels(df, target_x=150, stop_y=100, horizon_n=10)
    assert labels.iloc[0] == -1

def test_no_hit_within_horizon_produces_label_0():
    """Flat price series never touches either barrier -> label 0 (time barrier expiry)."""
    n = 20
    df = _flat_df(n)
    labels = generate_labels(df, target_x=150, stop_y=100, horizon_n=10)
    assert labels.iloc[0] == 0

def test_insufficient_future_data_produces_nan():
    """Rows near the end of the dataset (fewer than horizon_n future candles) must be NaN."""
    n = 20
    df = _flat_df(n)
    horizon = 10
    labels = generate_labels(df, target_x=150, stop_y=100, horizon_n=horizon)
    # Last `horizon` rows must all be NaN since we cannot look beyond the dataset
    assert labels.iloc[-horizon:].isna().all()
    # Rows before that with enough future data must NOT be NaN
    assert not labels.iloc[:n - horizon].isna().any()

def test_double_touch_same_candle_defaults_to_conservative_loss():
    """
    If a single future candle's range covers BOTH barriers (huge volatility candle),
    intra-candle order is unknowable from OHLC alone - the conservative assumption
    (stop hit first) must be applied, matching documented behavior.
    """
    n = 20
    df = _flat_df(n)
    df.loc[4, "high"] = 2000.0 + 200.0
    df.loc[4, "low"] = 2000.0 - 200.0
    labels = generate_labels(df, target_x=150, stop_y=100, horizon_n=10)
    assert labels.iloc[0] == -1

def test_label_distribution_on_mock_data_is_not_degenerate():
    """
    Statistical validation: on a reasonably long mock M15 series, the label
    distribution should not be 100% one class (which would indicate a bug or a
    misconfigured X/Y that makes one outcome nearly impossible).
    """
    df = fetch_mock_candles("M15", n_candles=1000, sessions_config=SESSIONS)
    labels = generate_labels_from_config(df, CFG)
    summary = label_distribution_summary(labels)
    assert summary["total_valid"] > 0
    # No single outcome class should dominate 100% - some variation expected even in mock data
    assert summary["pct_no_hit"] < 100.0 or summary["total_valid"] < 5

def test_generate_labels_from_config_uses_config_values():
    """Sanity check that the config wrapper actually threads X/Y/N from config.yaml."""
    df = fetch_mock_candles("M15", n_candles=100, sessions_config=SESSIONS)
    direct = generate_labels(
        df,
        target_x=CFG["labeling"]["target_pips_x"],
        stop_y=CFG["labeling"]["stop_pips_y"],
        horizon_n=CFG["labeling"]["horizon_candles_n"],
    )
    via_config = generate_labels_from_config(df, CFG)
    pd.testing.assert_series_equal(direct, via_config)

```

FILE: logs\\_\_init\_\_.py

# [EMPTY FILE]

FILE: logs\journal.py

```

"""
Signal journal: logs every signal generated by the pipeline to SQLite, including
inputs (regime, confidence, bias) and outcome (filled in later via update_outcome).

Two-phase logging:
1. log_signal()      - called immediately when a signal is generated (outcome unknown)
2. update_outcome() - called after the trade resolves (target/stop/timeout hit)

```

This separation mirrors real trading: you know the signal at entry time, but the outcome only at exit time. The journal is the ground truth for performance reporting.

```

"""
import sqlite3
import time
from datetime import datetime, timezone
from typing import Optional

CREATE_TABLE_SQL = """
CREATE TABLE IF NOT EXISTS signal_journal (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    generated_at   TEXT NOT NULL,
    timestamp_utc  INTEGER NOT NULL,
    session        TEXT,
    regime         TEXT,
    bias           TEXT NOT NULL,
    confidence     REAL NOT NULL,
    entry_zone_low REAL,
    entry_zone_high REAL,
    invalidation   REAL,
    target         REAL,
    reasoning      TEXT,
    outcome        TEXT,
    outcome_pnl    REAL,
    outcome_logged_at TEXT
);
"""

class SignalJournal:
    def __init__(self, db_path: str):
        self.db_path = db_path
        self._init_db()

    def _connect(self):
        return sqlite3.connect(self.db_path)

    def _init_db(self):
        with self._connect() as conn:
            conn.execute(CREATE_TABLE_SQL)
            conn.commit()

    def log_signal(self, signal: dict) -> int:
        """
        Insert a new signal row. Returns the new row id (used later for update_outcome).
        signal is the dict from realtime/pipeline.py::generate_signal().
        """
        entry_zone = signal.get("entry_zone") or [None, None]
        targets = signal.get("targets") or [None]
        row = (
            signal.get("generated_at", datetime.now(timezone.utc).isoformat()),
            int(signal.get("timestamp_utc", 0)),
            signal.get("session"),
            signal.get("regime"),
            signal["bias"],
            float(signal["confidence"]),
            entry_zone[0],
            entry_zone[1] if len(entry_zone) > 1 else None,
            signal.get("invalidation"),
            targets[0],
            signal.get("reasoning_summary"),
        )
        with self._connect() as conn:
            cursor = conn.execute(
                """INSERT INTO signal_journal
                (generated_at, timestamp_utc, session, regime, bias, confidence,
                 entry_zone_low, entry_zone_high, invalidation, target, reasoning)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                row,
            )
            conn.commit()
            return cursor.lastrowid

    def update_outcome(self, row_id: int, outcome: str, pnl: Optional[float] = None):
        """
        Fill in trade outcome after it resolves. outcome: "target", "stop", "timeout", "cancelled".
        """
        with self._connect() as conn:
            conn.execute(
                """UPDATE signal_journal
                SET outcome=?, outcome_pnl=?, outcome_logged_at=?
                WHERE id=?""",
                (outcome, pnl, datetime.now(timezone.utc).isoformat(), row_id),
            )
            conn.commit()

    def fetch_all(self) -> list:
        with self._connect() as conn:

```

```

        rows = conn.execute("SELECT * FROM signal_journal ORDER BY id").fetchall()
    return rows

```

```

def fetch_unresolved(self) -> list:
    """Returns all signals where outcome is still NULL (trade not yet closed)."""
    with self._connect() as conn:
        rows = conn.execute(
            "SELECT * FROM signal_journal WHERE outcome IS NULL ORDER BY id"
        ).fetchall()
    return rows

```

FILE: logs\reporter.py

```

"""
Daily/weekly summary report generator. Reads the signal journal SQLite DB and
produces a structured performance summary dict (and optionally a formatted string
for Telegram or stdout). Pure read-only – never writes to the journal.
"""

import sqlite3
import numpy as np
from datetime import datetime, timezone, timedelta
from typing import Optional

def _fetch_resolved(db_path: str, since_ts: Optional[str] = None) -> list:
    """Fetch all resolved (outcome not NULL) signal rows, optionally filtered by date."""
    query = "SELECT bias, confidence, session, regime, outcome, outcome_pnl FROM signal_journal WHERE outcome IS NOT NULL"
    params = []
    if since_ts:
        query += " AND generated_at >= ?"
        params.append(since_ts)
    with sqlite3.connect(db_path) as conn:
        return conn.execute(query, params).fetchall()

def generate_summary(db_path: str, since_ts: Optional[str] = None) -> dict:
    """
    Returns a performance summary dict over all resolved trades since since_ts.
    Keys: n_signals, n_resolved, win_rate, profit_factor, total_pnl, max_drawdown,
        by_session (dict of session -> {n, win_rate}), by_regime.
    """
    rows = _fetch_resolved(db_path, since_ts)
    if not rows:
        return {"n_resolved": 0, "win_rate": None, "profit_factor": None,
            "total_pnl": 0.0, "max_drawdown": 0.0, "by_session": {}, "by_regime": {}}

    pnls = [r[5] for r in rows if r[5] is not None]
    wins = [p for p in pnls if p > 0]
    losses = [p for p in pnls if p <= 0]

    win_rate = len(wins) / len(pnls) * 100 if pnls else None
    gross_profit = sum(wins)
    gross_loss = -sum(losses)
    profit_factor = gross_profit / gross_loss if gross_loss > 0 else float("inf")

    cum = np.cumsum(pnls)
    running_max = np.maximum.accumulate(cum)
    drawdowns = cum - running_max
    max_drawdown = float(drawdowns.min()) if len(drawdowns) > 0 else 0.0

    by_session = {}
    by_regime = {}
    for bias, conf, session, regime, outcome, pnl in rows:
        if pnl is None:
            continue
        for key, bucket in [(session, by_session), (regime, by_regime)]:
            if key not in bucket:
                bucket[key] = {"n": 0, "wins": 0, "total_pnl": 0.0}
            bucket[key]["n"] += 1
            bucket[key]["total_pnl"] += pnl
            if pnl > 0:
                bucket[key]["wins"] += 1

    for bucket in [by_session, by_regime]:
        for k in bucket:
            n = bucket[k]["n"]
            bucket[k]["win_rate"] = round(bucket[k]["wins"] / n * 100, 1) if n > 0 else None

    return {
        "n_resolved": len(pnls),
        "win_rate": round(win_rate, 1) if win_rate is not None else None,
        "profit_factor": round(profit_factor, 3),
        "total_pnl": round(sum(pnls), 4),
        "max_drawdown": round(max_drawdown, 4),
        "by_session": by_session,
        "by_regime": by_regime,
    }

```

```
def format_summary_message(summary: dict) -> str:
    """Formats the summary dict into a human-readable string for Telegram or stdout."""
    if summary["n_resolved"] == 0:
        return "XAUUSD Performance Report\nNo resolved trades yet."

    lines = [
        "XAUUSD Performance Report",
        f"Resolved trades : {summary['n_resolved']}",
        f"Win rate       : {summary['win_rate']}%",
        f"Profit factor    : {summary['profit_factor']}",
        f"Total PnL       : {summary['total_pnl']}",
        f"Max drawdown     : {summary['max_drawdown']}",
    ]
    if summary["by_session"]:
        lines.append("\nBy session:")
        for s, m in summary["by_session"].items():
            lines.append(f"    {s}: {m['n']} trades, {m['win_rate']}% win rate, PnL {round(m['total_pnl'],4)}")
    return "\n".join(lines)
```

FILE: logs\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: logs\tests\test\_journal.py

```
"""
Unit tests for logs/journal.py and logs/reporter.py.
Run with: pytest logs/tests/test_journal.py -v
"""
import os
import sys
import pytest
import tempfile

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from logs.journal import SignalJournal
from logs.reporter import generate_summary, format_summary_message

SAMPLE_SIGNAL = {
    "bias": "long",
    "confidence": 0.72,
    "entry_zone": [2400.0, 2402.0],
    "invalidation": 2390.0,
    "targets": [2420.0],
    "reasoning_summary": "regime=trend_up, rule_vote=1, ml_p_long=0.72",
    "regime": "trend_up",
    "timestamp_utc": 1700000000,
    "session": "london",
    "generated_at": "2024-01-01T10:00:00+00:00",
}

@pytest.fixture
def tmp_journal(tmp_path):
    return SignalJournal(str(tmp_path / "test_journal.sqlite"))

def test_log_signal_returns_id(tmp_journal):
    row_id = tmp_journal.log_signal(SAMPLE_SIGNAL)
    assert isinstance(row_id, int) and row_id > 0

def test_fetch_all_returns_logged_signal(tmp_journal):
    tmp_journal.log_signal(SAMPLE_SIGNAL)
    rows = tmp_journal.fetch_all()
    assert len(rows) == 1
    assert rows[0][5] == "long" # bias column

def test_update_outcome_fills_result(tmp_journal):
    row_id = tmp_journal.log_signal(SAMPLE_SIGNAL)
    tmp_journal.update_outcome(row_id, "target", pnl=150.0)
    rows = tmp_journal.fetch_all()
    assert rows[0][-3] == "target"
    assert rows[0][-2] == 150.0

def test_fetch_unresolved_excludes_resolved(tmp_journal):
    id1 = tmp_journal.log_signal(SAMPLE_SIGNAL)
    tmp_journal.log_signal(**SAMPLE_SIGNAL, "bias": "short")
    tmp_journal.update_outcome(id1, "stop", pnl=-100.0)
    unresolved = tmp_journal.fetch_unresolved()
    assert len(unresolved) == 1
    assert unresolved[0][5] == "short"
```



```

"""
Core ensemble logic - pure function, no side effects, fully deterministic given inputs.
Called once per row/candle by realtime/pipeline.py and backtest strategy wrappers.
"""

ens_cfg = cfg["ensemble"]
meta_cfg = cfg["model"]["meta_filter"]

rule_vote = rule_based_signal(regime)

# ML directional vote and its own confidence (distance from 0.5, rescaled to [0,1])
ml_vote = 1 if ml_p_long > ml_p_short else (-1 if ml_p_short > ml_p_long else 0)
ml_confidence = abs(ml_p_long - 0.5) * 2 # 0.5->0 conf, 0.0 or 1.0 -> full conf

rule_conf = _rule_confidence_component(rule_vote)

# Weighted blend of confidences, direction-aware: only blend confidence towards
# a direction if rule and ML AGREE on that direction: otherwise the disagreement
# itself is a signal of uncertainty and confidence is heavily penalized.
agree = (rule_vote == ml_vote) and rule_vote != 0

if agree:
    blended_confidence = (ens_cfg["weight_rule_based"] * rule_conf +
                          ens_cfg["weight_ml_probability"] * ml_confidence)
    final_vote = rule_vote
else:
    # Disagreement or one side neutral: confidence collapses toward the weaker signal.
    # This is intentional risk-aversion consistent with "fewer, higher-confidence alerts".
    blended_confidence = min(rule_conf, ml_confidence) * 0.5
    final_vote = ml_vote if ml_confidence > rule_conf else rule_vote

# Meta-filter: suppress signals in known choppy/noisy regimes unless confidence is
# exceptionally high (>= min_regime_confidence) - protects against overtrading ranges.
suppressed = False
if regime.value in meta_cfg["suppress_regimes"] and blended_confidence < meta_cfg["min_regime_confidence"]:
    suppressed = True
    blended_confidence = 0.0
    final_vote = 0

# No-trade gates: insufficient confidence, neutral vote, or explicit regime no_trade
if regime == RegimeLabel.NO_TRADE or final_vote == 0 or blended_confidence < ens_cfg["min_confidence_to_alert"]:
    bias = "no_trade"
elif final_vote == 1:
    bias = "long"
else:
    bias = "short"

reasoning = (
    f"regime={regime.value}, rule_vote={rule_vote}, ml_p_long={ml_p_long:.3f}, "
    f"ml_p_short={ml_p_short:.3f}, agree={agree}, suppressed_by_meta_filter={suppressed}, "
    f"blended_confidence={blended_confidence:.3f}"
)

return EnsembleSignal(
    bias=bias,
    confidence=round(float(blended_confidence), 4),
    rule_vote=rule_vote,
    ml_p_long=float(ml_p_long),
    ml_p_short=float(ml_p_short),
    regime=regime.value,
    suppressed_by_meta_filter=suppressed,
    reasoning_summary=reasoning,
)

```

FILE: model\ensemble\_backtest.py

```

"""
Ensemble-aware backtest wrapper: routes the ensemble's bias/confidence through the
existing EventDrivenBacktester execution logic (same signal-at-close(i) ->
entry-at-open(i+1) timing, same spread/slippage/one-position-at-a-time rules).
This is a thin adapter, not a duplicate engine - it reuses backtest/engine.py's
Trade dataclass and exit logic, only replacing WHICH signal drives entries.
"""

import numpy as np
import pandas as pd
from dataclasses import dataclass
from typing import Optional, List

from regime.classifier import RegimeLabel
from model.ensemble import compute_ensemble_signal
from backtest.engine import Trade

class EnsembleBacktester:
    """
    Same execution mechanics as EventDrivenBacktester, but the entry signal comes
    from the full ensemble (rule + ML + meta-filter) instead of rule_based signal alone.
    Requires df to already contain: regime (RegimeLabel), ml_p_long, ml_p_short columns,
    computed causally upstream (regime/classifier.py + model/predictor.py).
    """

```

```

"""

def __init__(self, cfg: dict):
    self.cfg = cfg
    bt_cfg = cfg["backtest"]
    lab_cfg = cfg["labeling"]
    self.spread = bt_cfg["spread_points"] / 100.0
    self.slippage = bt_cfg["slippage_points"] / 100.0
    self.balance = bt_cfg["initial_balance"]
    self.target_x = lab_cfg["target_pips_x"]
    self.stop_y = lab_cfg["stop_pips_y"]
    self.horizon_n = lab_cfg["horizon_candles_n"]
    self.trades: List[Trade] = []

def run(self, df: pd.DataFrame) -> List[Trade]:
    n = len(df)
    open_position: Optional[Trade] = None
    pending_direction: Optional[int] = None

    opens = df["open"].values
    highs = df["high"].values
    lows = df["low"].values
    closes = df["close"].values
    timestamps = df["timestamp_utc"].values
    sessions = df["session"].values
    regimes = df["regime"].values
    p_long = df["ml_p_long"].values
    p_shorts = df["ml_p_short"].values

    for i in range(n):
        if open_position is None and pending_direction is not None and pending_direction != 0:
            direction = pending_direction
            entry_price = opens[i] + (self.spread / 2 if direction == 1 else -self.spread / 2)
            stop_price = entry_price - direction * self.stop_y
            target_price = entry_price + direction * self.target_x
            open_position = Trade(
                entry_ts=int(timestamps[i]), entry_price=entry_price, direction=direction,
                stop_price=stop_price, target_price=target_price, session=str(sessions[i]),
                regime_at_entry=str(regimes[i - 1]) if i > 0 else str(regimes[i]),
            )
            pending_direction = None

        if open_position is not None:
            direction = open_position.direction
            hit_target = (highs[i] >= open_position.target_price) if direction == 1 else (lows[i] <= open_position.target_price)
            hit_stop = (lows[i] <= open_position.stop_price) if direction == 1 else (highs[i] >= open_position.stop_price)
            step = df["timestamp_utc"].diff().mode().iloc[0] if len(df) > 1 else 1
            candles_held = int((timestamps[i] - open_position.entry_ts) / step)

            exit_reason, exit_price = None, None
            if hit_target and hit_stop:
                exit_reason, exit_price = "stop", open_position.stop_price
            elif hit_stop:
                exit_reason, exit_price = "stop", open_position.stop_price
            elif hit_target:
                exit_reason, exit_price = "target", open_position.target_price
            elif candles_held >= self.horizon_n:
                exit_reason, exit_price = "timeout", closes[i]

            if exit_reason is not None:
                exit_price += (-self.slippage if direction == 1 else self.slippage)
                open_position.exit_ts = int(timestamps[i])
                open_position.exit_price = exit_price
                open_position.exit_reason = exit_reason
                open_position.pnl = direction * (exit_price - open_position.entry_price)
                self.balance += open_position.pnl
                self.trades.append(open_position)
                open_position = None

            if open_position is None:
                sig = compute_ensemble_signal(regimes[i], p_long[i], p_shorts[i], self.cfg)
                pending_direction = {"long": 1, "short": -1, "no_trade": 0}[sig.bias]

    return self.trades

```

FILE: model\predictor.py

```

"""
Model inference: load a trained/calibrated model and produce P(long)/P(short) for a
single row (or batch) of features at prediction time.

CRITICAL NO-LOOK-AHEAD NOTE:
predict() only ever consumes FEATURE_COLUMNS (causal, from features/) - it must NEVER
be given a row that also contains the 'label' column, since label is only meaningful
in hindsight (see model/trainer.py docstring). This module enforces that by explicitly
selecting only the trained feature_cols list saved alongside the model, ignoring any
extra columns (including 'label' if accidentally present) in the input DataFrame.
"""

```

```

import numpy as np
import pandas as pd

from model.trainer import load_model, FEATURE_COLUMNS

class ModelPredictor:
    """Wraps a loaded, calibrated model + its feature column list for safe inference."""

    def __init__(self, model_path: str):
        bundle = load_model(model_path)
        self.model = bundle["model"]
        self.feature_cols = bundle["feature_cols"]

    def predict_proba(self, row_or_df) -> pd.DataFrame:
        """
        Accepts a single pd.Series (one row) or a pd.DataFrame (batch).
        Returns a DataFrame with columns: p_long, p_short - guaranteed to sum to 1.0
        per row since this is a binary classifier (label 1 = long-favorable outcome).
        Any missing feature column raises explicitly rather than silently filling zeros,
        since silent zero-filling could produce a confidently wrong prediction.
        """
        if isinstance(row_or_df, pd.Series):
            df = row_or_df.to_frame().T
        else:
            df = row_or_df

        missing = set(self.feature_cols) - set(df.columns)
        if missing:
            raise ValueError(f"Missing required feature columns for inference: {missing}")

        X = df[self.feature_cols]
        if X.isnull().any().any():
            raise ValueError("NaN found in feature inputs at inference time - cannot predict on incomplete data.")

        X = X.astype(float)
        proba = self.model.predict_proba(X) # columns: [P(class=0), P(class=1)] = [P(short), P(long)]
        return pd.DataFrame({"p_short": proba[:, 0], "p_long": proba[:, 1]}, index=df.index)

    def predict_single(self, row: pd.Series) -> dict:
        """Convenience wrapper returning a plain dict for the single-row realtime use case."""
        result = self.predict_proba(row)
        return {"p_long": float(result["p_long"].iloc[0]), "p_short": float(result["p_short"].iloc[0])}

```

FILE: model\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: model\tests\test\_ensemble.py

```

"""
Unit tests for model/ensemble.py.
Run with: pytest model/tests/test_ensemble.py -v
"""

import os
import sys

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from regime.classifier import RegimeLabel
from model.ensemble import compute_ensemble_signal, EnsembleSignal

CFG = load_config()

def test_strong_agreement_produces_high_confidence_long():
    """Rule says TREND_UP (vote=+1) and ML strongly agrees (p_long=0.9) -> confident long."""
    sig = compute_ensemble_signal(RegimeLabel.TREND_UP, ml_p_long=0.9, ml_p_short=0.1, cfg=CFG)
    assert sig.bias == "long"
    assert sig.confidence > 0.5
    assert isinstance(sig, EnsembleSignal)

def test_strong_agreement_produces_high_confidence_short():
    sig = compute_ensemble_signal(RegimeLabel.TREND_DOWN, ml_p_long=0.1, ml_p_short=0.9, cfg=CFG)
    assert sig.bias == "short"
    assert sig.confidence > 0.5

def test_disagreement_collapses_confidence_to_no_trade():
    """Rule says TREND_UP but ML strongly favors short -> disagreement, confidence collapses."""
    sig = compute_ensemble_signal(RegimeLabel.TREND_UP, ml_p_long=0.15, ml_p_short=0.85, cfg=CFG)
    assert sig.bias == "no_trade"
    assert sig.confidence < CFG["ensemble"]["min_confidence_to_alert"]

def test_regime_no_trade_always_forces_no_trade_bias():

```



```

    """Even with maximally confident ML probabilities, NO_TRADE regime must override everything."""
    sig = compute_ensemble_signal(RegimeLabel.NO_TRADE, ml_p_long=0.99, ml_p_short=0.01, cfg=CFG)
    assert sig.bias == "no_trade"

def test_meta_filter_suppresses_low_confidence_in_choppy_regime():
    """
    RANGE and COMPRESSION are in suppress_regimes by default (config.yaml).
    Moderate agreement/confidence below min_regime_confidence must be suppressed.
    """
    sig = compute_ensemble_signal(RegimeLabel.RANGE, ml_p_long=0.6, ml_p_short=0.4, cfg=CFG)
    assert sig.suppressed_by_meta_filter is True
    assert sig.bias == "no_trade"
    assert sig.confidence == 0.0

def test_meta_filter_allows_exceptionally_high_confidence_in_choppy_regime():
    """
    Even in a suppress_regimes regime, sufficiently high blended confidence (>= min_regime_confidence)
    should NOT be suppressed - the gate only fires when confidence is ALSO low.
    """
    sig = compute_ensemble_signal(RegimeLabel.RANGE, ml_p_long=0.95, ml_p_short=0.05, cfg=CFG)
    # Rule vote is 0 in RANGE regime (rule_based_signal only fires on trend regimes),
    # so agree=False by construction -> confidence collapses regardless of ML strength.
    # This IS the correct, conservative behavior: rule-based baseline never votes in RANGE,
    # so the ensemble cannot reach high confidence purely from ML in a choppy regime.
    assert sig.bias == "no_trade"

def test_confidence_never_exceeds_one():
    sig = compute_ensemble_signal(RegimeLabel.TREND_UP, ml_p_long=1.0, ml_p_short=0.0, cfg=CFG)
    assert 0.0 <= sig.confidence <= 1.0

def test_reasoning_summary_is_nonempty_string():
    sig = compute_ensemble_signal(RegimeLabel.TREND_UP, ml_p_long=0.8, ml_p_short=0.2, cfg=CFG)
    assert isinstance(sig.reasoning_summary, str)
    assert len(sig.reasoning_summary) > 10

def test_below_min_confidence_threshold_forces_no_trade():
    """Even with rule+ML agreement, if blended confidence < min_confidence_to_alert, must be no_trade."""
    # Weak ML confidence (p_long just above 0.5) combined with rule agreement may still
    # fall below threshold depending on weights - verify the gate is respected either way.
    sig = compute_ensemble_signal(RegimeLabel.TREND_UP, ml_p_long=0.52, ml_p_short=0.48, cfg=CFG)
    if sig.confidence < CFG["ensemble"]["min_confidence_to_alert"]:
        assert sig.bias == "no_trade"

```

FILE: model\tests\test\_trainer.py

```

"""
Unit tests for model/trainer.py and model/predictor.py.
Run with: pytest model/tests/test_trainer.py -v
"""

import os
import sys
import numpy as np
import pandas as pd
import pytest

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import fetch_mock_candles
from features.indicators import build_all_indicators
from features.candle_anatomy import candle_anatomy
from regime.classifier import add_regime_indicators
from labeling.label_generator import generate_labels_from_config
from model.trainer import build_training_matrix, time_ordered_split, train_model, calibrate_model, save_model, load_model, FEATURE_COLUMNS
from model.predictor import ModelPredictor

CFG = load_config()
SESSIONS = CFG["sessions"]

# Override labeling barriers to match mock data scale (~0.15 pts/candle drift)
import copy
CFG = copy.deepcopy(CFG)
CFG["labeling"]["target_pips_x"] = 3.0
CFG["labeling"]["stop_pips_y"] = 2.0

def _full_featured_labeled_df(n=3000, seed=55):
    df = fetch_mock_candles("M15", n_candles=n, sessions_config=SESSIONS, seed=seed)
    df = build_all_indicators(df, CFG)
    df = candle_anatomy(df)
    df = add_regime_indicators(df, CFG)
    df["mtf_confluence_score"] = 0.0 # placeholder in isolated test - real value comes from mtf_confluence.py in the full pipeline
    labels = generate_labels_from_config(df, CFG)

```

```

df["label"] = labels
return df

def test_build_training_matrix_drops_nan_and_zero_labels():
    df = _full_featured_labeled_df()
    X, y, cols = build_training_matrix(df)
    assert len(X) == len(y)
    assert set(y.unique()) <= {0, 1}
    assert not X.isnull().any().any()
    assert len(X) < len(df) # some rows dropped (NaN warmup rows, label==0 rows, NaN label tail)

def test_time_ordered_split_preserves_order():
    df = _full_featured_labeled_df()
    X, y, cols = build_training_matrix(df)
    X_train, X_test, y_train, y_test = time_ordered_split(X, y, train_ratio=0.8)
    assert X_train.index.max() < X_test.index.min(), "Train indices must all precede test indices (no shuffling)"
    assert len(X_train) + len(X_test) == len(X)

def test_train_and_predict_end_to_end(tmp_path):
    df = _full_featured_labeled_df(n=4000, seed=77)
    X, y, cols = build_training_matrix(df)
    X_train, X_test, y_train, y_test = time_ordered_split(X, y, train_ratio=0.8)

    if len(X_train) < 30 or len(X_test) < 5 or y_train.nunique() < 2:
        pytest.skip("Insufficient class diversity in mock data for this seed - not a code defect")

    base_model = train_model(X_train, y_train, CFG)
    calibrated = calibrate_model(base_model, X_train, y_train, CFG)

    model_path = str(tmp_path / "test_model.joblib")
    save_model(calibrated, cols, model_path)

    predictor = ModelPredictor(model_path)
    preds = predictor.predict_proba(X_test)

    assert len(preds) == len(X_test)
    assert np.allclose(preds["p_long"] + preds["p_short"], 1.0, atol=1e-6)
    assert (preds["p_long"] >= 0).all() and (preds["p_long"] <= 1).all()

def test_predictor_raises_on_missing_feature_columns(tmp_path):
    df = _full_featured_labeled_df(n=2000, seed=99)
    X, y, cols = build_training_matrix(df)
    if y.nunique() < 2 or len(X) < 30:
        pytest.skip("Insufficient class diversity in mock data for this seed - not a code defect")

    base_model = train_model(X, y, CFG)
    calibrated = calibrate_model(base_model, X, y, CFG)
    model_path = str(tmp_path / "test_model2.joblib")
    save_model(calibrated, cols, model_path)

    predictor = ModelPredictor(model_path)
    incomplete_row = X.iloc[0].drop(cols[0]) # remove one required feature
    with pytest.raises(ValueError):
        predictor.predict_proba(incomplete_row)

def test_predictor_raises_on_nan_input(tmp_path):
    df = _full_featured_labeled_df(n=2000, seed=123)
    X, y, cols = build_training_matrix(df)
    if y.nunique() < 2 or len(X) < 30:
        pytest.skip("Insufficient class diversity in mock data for this seed - not a code defect")

    base_model = train_model(X, y, CFG)
    calibrated = calibrate_model(base_model, X, y, CFG)
    model_path = str(tmp_path / "test_model3.joblib")
    save_model(calibrated, cols, model_path)

    predictor = ModelPredictor(model_path)
    bad_row = X.iloc[0].copy()
    bad_row[cols[0]] = np.nan
    with pytest.raises(ValueError):
        predictor.predict_proba(bad_row)

```

FILE: model\trainer.py

"""

Gradient boosting model trainer (XGBoost) for directional bias prediction.

CRITICAL NO-LOOK-AHEAD DESIGN:

Training data (X, y) pairs are built by joining features/ (computed causally, see features/tests/test\_no\_lookahead.py) with labeling/ (which is INTENTIONALLY forward-looking, see labeling/label\_generator.py docstring). The join is valid for TRAINING purposes only: row i's features (known at close of candle i) are paired with row i's label (the future outcome after candle i) - this is exactly how supervised learning

is supposed to work. The critical invariant is that this label column must NEVER be fed back into features/ or regime/ as an input, and must NEVER be available to realtime/pipeline.py at inference time (it doesn't exist yet, by construction, live).

Train/test split is TIME-ORDERED (not shuffled) to respect temporal causality - shuffling would let the model implicitly learn from "future" rows relative to some test rows, which is a subtle but real leakage risk in time series ML.

Probability calibration (isotonic or sigmoid) is applied on top of raw XGBoost probabilities because tree ensembles are often poorly calibrated out-of-the-box - calibration is essential here because the ensemble layer (Step 9) and alert thresholds rely on P(long)/P(short) being genuinely interpretable probabilities, not just rank-ordered scores.

```
"""
import numpy as np
import pandas as pd
import xgboost as xgb
from sklearn.calibration import CalibratedClassifierCV
from sklearn.base import BaseEstimator, ClassifierMixin
import joblib
import os

FEATURE_COLUMNS = [
    "ema_9", "ema_21", "ema_50", "ema_200", "rsi", "macd_line", "macd_signal", "macd_hist",
    "atr", "bb_width", "body_ratio", "upper_wick_ratio", "lower_wick_ratio", "candle_direction",
    "adx", "plus_di", "minus_di", "atr_ratio", "mtf_confluence_score",
]
```

```
def build_training_matrix(df: pd.DataFrame, label_col: str = "label") -> tuple:
    """
    Extracts (X, y) from a fully-featured, labeled DataFrame.
    Drops rows with NaN in required feature columns or NaN label (label is NaN
    near the end of the dataset where insufficient future data existed, per
    labeling/label_generator.py - these rows are correctly excluded from training).
    Only binary direction is modeled here (label 0 = "no clear outcome" is dropped
    from the classifier's training set; the meta-filter/ensemble in Step 9 handles
    "no signal" via regime + confidence gating, not via a 3rd model class).
    """
    available_cols = [c for c in FEATURE_COLUMNS if c in df.columns]
    working = df[available_cols + [label_col]].copy()
    working = working[working[label_col].isin([1, -1, 1.0, -1.0])] # binary: drop 0/NaN outcomes (float-safe)
    working = working.dropna()

    X = working[available_cols]
    y = (working[label_col] == 1).astype(int) # 1 = upper hit (long-favorable), 0 = lower hit
    return X, y, available_cols
```

```
def time_ordered_split(X: pd.DataFrame, y: pd.Series, train_ratio: float) -> tuple:
    """Split strictly by row order (time-ordered), never shuffled, to avoid temporal leakage."""
    n = len(X)
    split_idx = int(n * train_ratio)
    X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
    y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]
    return X_train, X_test, y_train, y_test
```

```
def train_model(X_train: pd.DataFrame, y_train: pd.Series, cfg: dict):
    """Train raw XGBoost classifier with config-driven hyperparameters/seed."""
    model_cfg = cfg["model"]
    clf = xgb.XGBClassifier(
        n_estimators=200,
        max_depth=4,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=model_cfg["random_seed"],
        eval_metric="logloss",
        use_label_encoder=False if hasattr(xgb.XGBClassifier(), "use_label_encoder") else None,
    )
    clf.fit(X_train, y_train)
    return clf
```

```
def calibrate_model(base_model, X_train: pd.DataFrame, y_train: pd.Series, cfg: dict):
    """
    Wrap the trained model with probability calibration.
    method comes from config.yaml model.calibration_method ("isotonic" or "sigmoid").
    cv="prefit" would require a held-out calibration set; here we use cv=3 with
    time-respecting behavior approximated by NOT shuffling (sklearn CalibratedClassifierCV
    internally does K-fold, which for small time series windows is an acceptable
    approximation at this training-set scale, but documented as a simplification).
    """
    method = cfg["model"]["calibration_method"]
    calibrated = CalibratedClassifierCV(base_model, method=method, cv=3)
```

```
calibrated.fit(X_train, y_train)
return calibrated
```

```
def save_model(model, feature_cols: list, path: str):
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    joblib.dump({"model": model, "feature_cols": feature_cols}, path)
```

```
def load_model(path: str):
    return joblib.load(path)
```

FILE: pyproject.toml

```
[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["."]
```

FILE: realtime\\_\_init\_\_.py

```
# [EMPTY FILE]
```

FILE: realtime/app.py

```
"""
FastAPI inference service exposing the /signal endpoint.

Design decision: the RealtimePipeline is instantiated ONCE at app startup (loading
the model from disk a single time) rather than per-request, since model loading is
comparatively expensive and the model is immutable during the service's lifetime.
Config is also loaded once and shared - config.loader already caches it globally.

No secrets are read here directly; MODEL_PATH is the only path-like env var and it
points to a local file, not a credential.
"""
import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional, List

from config.loader import load_config, get_env
from realtime.pipeline import RealtimePipeline

app = FastAPI(title="XAUUSD Predictive Alert System", version="0.1.0")

CFG = load_config()
MODEL_PATH = get_env("MODEL_PATH", default=None)
DATA_MODE = get_env("DATA_MODE", default="mock") # "mock" or "live" - controlled via env, never hardcoded

pipeline = RealtimePipeline(cfg=CFG, model_path=MODEL_PATH, data_mode=DATA_MODE)

class SignalResponse(BaseModel):
    bias: str
    confidence: float
    entry_zone: Optional[List[float]] = None
    invalidation: Optional[float] = None
    targets: Optional[List[float]] = None
    reasoning_summary: str
    regime: str
    timestamp_utc: int
    session: str
    generated_at: str

@app.get("/health")
def health():
    """Basic liveness check - does not run the full pipeline."""
    return {"status": "ok", "data_mode": DATA_MODE, "model_loaded": pipeline._predictor is not None}

@app.get("/signal", response_model=SignalResponse)
def get_signal(n_candles: int = 300):
    """
    Runs the full pipeline (data -> features -> regime -> model -> ensemble) and
    returns the structured signal JSON. n_candles controls how much history is
    pulled for feature warm-up (must exceed regime.min_candles_for_regime + buffer).
    """
    try:
        result = pipeline.generate_signal(n_candles=n_candles)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Signal generation failed: {str(e)}")
```

FILE: realtime/pipeline.py

```
"""
Realtime inference pipeline: wires data -> features -> regime -> model -> ensemble
```

into a single callable that produces the structured signal JSON required by the FastAPI service (realtime/app.py).

#### CRITICAL NO-LOOK-AHEAD NOTE:

This pipeline operates on LIVE data only - the latest N candles up to "now". Every sub-step (features/, regime/, model/) was already proven causal in earlier steps' tests. This module adds ONE more guarantee: it only ever computes a signal for the LAST row of the fetched candle series (the most recently closed candle), never for an in-progress/incomplete candle, and never using any candle timestamped after "now".

"""

import time

from datetime import datetime, timezone

import pandas as pd

from config.loader import load\_config

from data.ingestion import fetch\_candles

from features.indicators import build\_all\_indicators

from features.candle\_anatomy import candle\_anatomy

from features.structure import detect\_structure

from regime.classifier import add\_regime\_indicators, classify\_regime\_series, RegimeLabel

from model.predictor import ModelPredictor

from model.ensemble import compute\_ensemble\_signal

class RealtimePipeline:

def \_\_init\_\_(self, asset\_key: str, cfg: dict = None, data\_mode: str = "live"):

self.cfg = cfg or load\_config()

self.asset\_key = asset\_key

self.data\_mode = data\_mode

assets = self.cfg["assets"]

if asset\_key not in assets:

raise ValueError(f"Unknown asset: {asset\_key}")

self.asset\_cfg = assets[asset\_key]

self.model\_path = self.asset\_cfg["model\_path"]

self.\_predictor = ModelPredictor(self.model\_path) if os.path.exists(self.model\_path) else None

def \_build\_features(self, df: pd.DataFrame) -> pd.DataFrame:

df = build\_all\_indicators(df, self.cfg)

df = candle\_anatomy(df)

df = detect\_structure(df, lookback=self.cfg["features"]["structure\_lookback"])

df = add\_regime\_indicators(df, self.cfg)

df["mtf\_confluence\_score"] = 0.0 # TODO: wire real MTF frames once multi-timeframe fetch is added to /signal

return df

def generate\_signal(self, n\_candles: int = 300) -> dict:

df = fetch\_candles(

asset\_key=self.asset\_key,

timeframe=timeframe,

n\_candles=n\_candles,

sessions\_cfg=sessions\_cfg,

mode=self.data\_mode,

)

if self.\_predictor is not None:

feature\_row = latest[self.\_predictor.feature\_cols]

if feature\_row.isnull().any():

# Warm-up / insufficient data - explicit no-trade, never guess

return self.\_no\_trade\_response(latest, regime, "Insufficient feature data (warm-up period)")

proba = self.\_predictor.predict\_single(feature\_row)

ml\_p\_long, ml\_p\_short = proba["p\_long"], proba["p\_short"]

else:

# No trained model wired yet - fall back to neutral ML probability (0.5/0.5),

# which the ensemble will treat as zero ML confidence, relying on rule-based only.

ml\_p\_long, ml\_p\_short = 0.5, 0.5

signal = compute\_ensemble\_signal(regime, ml\_p\_long, ml\_p\_short, self.cfg)

entry\_price = float(latest["close"])

atr\_val = float(latest["atr"]) if not pd.isna(latest["atr"]) else 0.0

target\_x = self.cfg["labeling"]["target\_pips\_x"]

stop\_y = self.cfg["labeling"]["stop\_pips\_y"]

if signal.bias == "long":

entry\_zone = [round(entry\_price - atr\_val \* 0.1, 2), round(entry\_price + atr\_val \* 0.1, 2)]

invalidation = round(entry\_price - stop\_y, 2)

targets = [round(entry\_price + target\_x, 2)]

elif signal.bias == "short":

entry\_zone = [round(entry\_price - atr\_val \* 0.1, 2), round(entry\_price + atr\_val \* 0.1, 2)]

invalidation = round(entry\_price + stop\_y, 2)

targets = [round(entry\_price - target\_x, 2)]

else:

entry\_zone, invalidation, targets = None, None, None

return {

```

        "bias": signal.bias,
        "confidence": signal.confidence,
        "entry_zone": entry_zone,
        "invalidation": invalidation,
        "targets": targets,
        "reasoning_summary": signal.reasoning_summary,
        "regime": regime.value,
        "timestamp_utc": int(latest["timestamp_utc"]),
        "session": str(latest["session"]),
        "generated_at": datetime.now(timezone.utc).isoformat(),
    }

def _no_trade_response(self, latest, regime, reason: str) -> dict:
    return {
        "bias": "no_trade",
        "confidence": 0.0,
        "entry_zone": None,
        "invalidation": None,
        "targets": None,
        "reasoning_summary": reason,
        "regime": regime.value if isinstance(regime, RegimeLabel) else str(regime),
        "timestamp_utc": int(latest["timestamp_utc"]),
        "session": str(latest["session"]),
        "generated_at": datetime.now(timezone.utc).isoformat(),
    }

```

FILE: realtime\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: realtime\tests\test\_pipeline.py

```

"""
Unit tests for realtime/pipeline.py.
Run with: pytest realtime/tests/test_pipeline.py -v
"""
import os
import sys

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from realtime.pipeline import RealtimePipeline

CFG = load_config()

def test_pipeline_generates_valid_signal_without_model():
    """Without a loaded ML model, pipeline must still work using neutral ML probs (rule-based only)."""
    pipeline = RealtimePipeline(cfg=CFG, model_path=None, data_mode="mock")
    result = pipeline.generate_signal(n_candles=300)

    assert result["bias"] in ("long", "short", "no_trade")
    assert 0.0 <= result["confidence"] <= 1.0
    assert isinstance(result["reasoning_summary"], str)
    assert "timestamp_utc" in result
    assert "session" in result

def test_pipeline_no_trade_has_null_trade_fields():
    """When bias is no_trade, entry_zone/invalidation/targets must be None, never fabricated."""
    pipeline = RealtimePipeline(cfg=CFG, model_path=None, data_mode="mock")
    result = pipeline.generate_signal(n_candles=300)
    if result["bias"] == "no_trade":
        assert result["entry_zone"] is None
        assert result["invalidation"] is None
        assert result["targets"] is None

def test_pipeline_directional_bias_has_populated_trade_fields():
    """When bias is long/short, entry_zone/invalidation/targets must all be populated."""
    pipeline = RealtimePipeline(cfg=CFG, model_path=None, data_mode="mock")
    # Run multiple times with different seeds is not directly supported (mock uses time-based seed
    # by default in fetch_candles unless seed is fixed) - so we just validate the contract if a
    # directional bias happens to occur, rather than forcing one artificially.
    for _ in range(5):
        result = pipeline.generate_signal(n_candles=300)
        if result["bias"] in ("long", "short"):
            assert result["entry_zone"] is not None and len(result["entry_zone"]) == 2
            assert result["invalidation"] is not None
            assert result["targets"] is not None and len(result["targets"]) >= 1
            break

def test_pipeline_uses_only_last_closed_candle():
    """The signal's timestamp must match the LAST row of the fetched series, never an earlier one."""
    pipeline = RealtimePipeline(cfg=CFG, model_path=None, data_mode="mock")
    from data.ingestion import fetch_mock_candles

```

```

df_check = fetch_mock_candles(CFG["labeling"]["labeling_timeframe"], n_candles=300, sessions_config=CFG["sessions"])
result = pipeline.generate_signal(n_candles=300)
# Timestamps are time-based (not seeded), so we just check the signal timestamp is recent/valid
assert result["timestamp_utc"] > 0

```

```

def test_pipeline_regime_field_is_valid_enum_value():
    from regime.classifier import RegimeLabel
    pipeline = RealtimePipeline(cfg=CFG, model_path=None, data_mode="mock")
    result = pipeline.generate_signal(n_candles=300)
    assert result["regime"] in [r.value for r in RegimeLabel]

```

FILE: regime\\_\_init\_\_.py

```
# [EMPTY FILE]
```

FILE: regime\classifier.py

```
"""
```

```
Rule-based market regime classifier.
```

```
Regimes: trend-up, trend-down, range, compression, reversal-watch, no-trade.
```

```
Design decision: this is a pure function of the LATEST row's feature values
(computed causally upstream in features/). No regime logic here looks at
future rows - it only reads columns already present on row i.
```

```
An ML-ready interface is defined separately in regime/ml_interface.py so this
rule-based classifier can be swapped for a trained classifier later without
changing the contract that regime/classifier.py -> RegimeLabel.
```

```
All thresholds come from config.yaml under `regime:` - nothing hardcoded here.
```

```
"""
```

```
from enum import Enum
```

```
import numpy as np
```

```
import pandas as pd
```

```
class RegimeLabel(str, Enum):
```

```
    TREND_UP = "trend_up"
```

```
    TREND_DOWN = "trend_down"
```

```
    RANGE = "range"
```

```
    COMPRESSION = "compression"
```

```
    REVERSAL_WATCH = "reversal_watch"
```

```
    NO_TRADE = "no_trade"
```

```
def _compute_adx(df: pd.DataFrame, period: int = 14) -> pd.Series:
```

```
    """
```

```
    Standard ADX (Average Directional Index) - causal by construction (rolling/ewm only).
```

```
    Included here rather than features/indicators.py because ADX is used exclusively
    for regime detection, not as a general-purpose feature for the ML model.
```

```
    """
```

```
    high, low, close = df["high"], df["low"], df["close"]
```

```
    up_move = high.diff()
```

```
    down_move = -low.diff()
```

```
    plus_dm = np.where((up_move > down_move) & (up_move > 0), up_move, 0.0)
```

```
    minus_dm = np.where((down_move > up_move) & (down_move > 0), down_move, 0.0)
```

```
    prev_close = close.shift(1)
```

```
    tr = pd.concat([
```

```
        high - low,
```

```
        (high - prev_close).abs(),
```

```
        (low - prev_close).abs(),
```

```
    ], axis=1).max(axis=1)
```

```
    atr_smooth = tr.ewm(alpha=1 / period, adjust=False, min_periods=period).mean()
```

```
    plus_di = 100 * pd.Series(plus_dm, index=df.index).ewm(alpha=1 / period, adjust=False, min_periods=period).mean() /
```

```
    atr_smooth.replace(0, np.nan)
```

```
    minus_di = 100 * pd.Series(minus_dm, index=df.index).ewm(alpha=1 / period, adjust=False, min_periods=period).mean() /
```

```
    atr_smooth.replace(0, np.nan)
```

```
    dx = 100 * (plus_di - minus_di).abs() / (plus_di + minus_di).replace(0, np.nan)
```

```
    adx = dx.ewm(alpha=1 / period, adjust=False, min_periods=period).mean()
```

```
    return adx.fillna(0.0), plus_di.fillna(0.0), minus_di.fillna(0.0)
```

```
def add_regime_indicators(df: pd.DataFrame, cfg: dict) -> pd.DataFrame:
```

```
    """Adds adx, plus_di, minus_di, bb_width_percentile, atr_ratio columns needed for classification."""
```

```
    out = df.copy()
```

```
    adx_period = cfg["features"]["atr_period"] # reuse ATR period config for ADX smoothing consistency
```

```
    adx, plus_di, minus_di = _compute_adx(out, adx_period)
```

```
    out["adx"] = adx
```

```
    out["plus_di"] = plus_di
```

```
    out["minus_di"] = minus_di
```

```
# bb_width must already exist (from features/indicators.py::build_all_indicators)
```

```

if "bb_width" in out.columns:
    rolling = out["bb_width"].rolling(window=50, min_periods=20)
    out["bb_width_percentile"] = rolling.apply(
        lambda x: float(pd.Series(x).rank(pct=True).iloc[-1] * 100), raw=False
    )
else:
    out["bb_width_percentile"] = np.nan

if "atr" in out.columns:
    atr_rolling_mean = out["atr"].rolling(window=50, min_periods=10).mean()
    out["atr_ratio"] = out["atr"] / atr_rolling_mean.replace(0, np.nan)
else:
    out["atr_ratio"] = np.nan

return out

def classify_regime_row(row: pd.Series, cfg: dict) -> RegimeLabel:
    """
    Classify a single row (Series with all indicator + regime columns) into a RegimeLabel.
    Order of checks matters: no_trade and reversal_watch gates are checked FIRST
    because they represent risk-off conditions that should override a trend/range call.
    """
    regime_cfg = cfg["regime"]

    price = row.get("close", np.nan)
    atr_val = row.get("atr", np.nan)

    # Gate 1: insufficient data / warm-up period -> no_trade
    if pd.isna(row.get("adx")) or pd.isna(atr_val) or pd.isna(price):
        return RegimeLabel.NO_TRADE

    # Gate 2: volatility floor - if ATR/price ratio too low, spreads/slippage would dominate any edge
    if price > 0 and (atr_val / price) < regime_cfg["no_trade_volatility_floor"]:
        return RegimeLabel.NO_TRADE

    # Gate 3: volatility spike -> reversal-watch (market is unstable, wait for confirmation)
    atr_ratio = row.get("atr_ratio", np.nan)
    if not pd.isna(atr_ratio) and atr_ratio > regime_cfg["atr_spike_multiplier"]:
        return RegimeLabel.REVERSAL_WATCH

    # Gate 4: compression - Bollinger width in bottom percentile -> squeeze, breakout pending
    bb_pctile = row.get("bb_width_percentile", np.nan)
    if not pd.isna(bb_pctile) and bb_pctile < regime_cfg["bb_width_compression_pctile"]:
        return RegimeLabel.COMPRESSION

    # Gate 5: trend detection via ADX + DI direction
    adx_val = row.get("adx", 0.0)
    plus_di = row.get("plus_di", 0.0)
    minus_di = row.get("minus_di", 0.0)

    if adx_val >= regime_cfg["adx_trend_threshold"]:
        if plus_di > minus_di:
            return RegimeLabel.TREND_UP
        elif minus_di > plus_di:
            return RegimeLabel.TREND_DOWN

    # Default: no strong trend, no compression, no volatility spike -> range-bound
    return RegimeLabel.RANGE

def classify_regime_series(df: pd.DataFrame, cfg: dict) -> pd.Series:
    min_candles = cfg["regime"]["min_candles_for_regime"]
    raw_labels = [classify_regime_row(row, cfg) for _, row in df.iterrows()]
    labels = pd.Series(raw_labels, index=df.index, dtype=object)
    labels.iloc[:min_candles] = RegimeLabel.NO_TRADE
    return labels

```

FILE: regime/ml\_interface.py

```

"""
Abstract interface for a future ML-based regime classifier.

Design decision: the rule-based classifier ships first (regime/classifier.py).
This interface defines the CONTRACT any future ML regime model must satisfy so
it can be swapped in without touching regime/classifier.py callers (labeling/,
backtest/, realtime/pipeline.py).

Contract: .classify(row: pd.Series) -> RegimeLabel
         .classify_series(df: pd.DataFrame) -> pd.Series[RegimeLabel]

Any concrete ML implementation (e.g. a trained sklearn/XGBoost multi-class model)
should subclass RegimeClassifierBase and implement these two methods.
"""
from abc import ABC, abstractmethod
import pandas as pd

from regime.classifier import RegimeLabel

```



```

class RegimeClassifierBase(ABC):
    """Abstract base - both rule-based and future ML regime classifiers implement this."""

    @abstractmethod
    def classify(self, row: pd.Series) -> RegimeLabel:
        """Classify a single row into a RegimeLabel."""
        raise NotImplementedError

    @abstractmethod
    def classify_series(self, df: pd.DataFrame) -> pd.Series:
        """Classify an entire DataFrame, row by row, returning a Series of RegimeLabel."""
        raise NotImplementedError

class RuleBasedRegimeClassifier(RegimeClassifierBase):
    """Adapter wrapping the existing rule-based functions to satisfy the ABC contract."""

    def __init__(self, cfg: dict):
        self.cfg = cfg

    def classify(self, row: pd.Series) -> RegimeLabel:
        from regime.classifier import classify_regime_row
        return classify_regime_row(row, self.cfg)

    def classify_series(self, df: pd.DataFrame) -> pd.Series:
        from regime.classifier import classify_regime_series
        return classify_regime_series(df, self.cfg)

class MLRegimeClassifierStub(RegimeClassifierBase):
    """
    Placeholder for a future trained ML regime classifier.
    NOT implemented yet per project plan (Step 4 = rule-based only, ML comes in Step 8+).
    Raises NotImplementedError intentionally so it fails loudly if wired in prematurely.
    """

    def __init__(self, model_path: str = None):
        self.model_path = model_path
        self.model = None # would be loaded from model_path in a real implementation

    def classify(self, row: pd.Series) -> RegimeLabel:
        raise NotImplementedError("ML regime classifier not yet trained - use RuleBasedRegimeClassifier.")

    def classify_series(self, df: pd.DataFrame) -> pd.Series:
        raise NotImplementedError("ML regime classifier not yet trained - use RuleBasedRegimeClassifier.")

```

FILE: regime\tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: regime\tests\test\_classifier.py

```

"""
Unit tests for regime/classifier.py.
Run with: pytest regime/tests/test_classifier.py -v
"""

import os
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.ingestion import fetch_mock_candles
from features.indicators import build_all_indicators
from regime.classifier import add_regime_indicators, classify_regime_series, classify_regime_row, RegimeLabel
from regime.ml_interface import RuleBasedRegimeClassifier, MLRegimeClassifierStub

CFG = load_config()
SESSIONS = CFG["sessions"]

def _prepared_df(n=300):
    df = fetch_mock_candles("M15", n_candles=n, sessions_config=SESSIONS)
    df = build_all_indicators(df, CFG)
    df = add_regime_indicators(df, CFG)
    return df

def test_warmup_period_forces_no_trade():
    df = _prepared_df(n=300)
    labels = classify_regime_series(df, CFG)
    min_candles = CFG["regime"]["min_candles_for_regime"]
    assert (labels.iloc[:min_candles] == RegimeLabel.NO_TRADE).all()

```

```

def test_classify_returns_valid_enum_values():
    df = _prepared_df(n=300)
    labels = classify_regime_series(df, CFG)
    valid_labels = set(RegimeLabel)
    assert set(labels.unique()) <= valid_labels

def test_synthetic_strong_uptrend_classified_as_trend_up():
    """Construct a deterministic strong uptrend and verify classifier picks it up."""
    n = 250
    ts = np.arange(n) * 900
    rng = np.random.default_rng(42)
    noise = rng.normal(0, 1.5, n)
    close = 2000 + np.arange(n) * 3.0 + noise
    df = pd.DataFrame({
        "timestamp_utc": ts,
        "open": close - 0.5,
        "high": close + np.abs(rng.normal(1.0, 0.3, n)),
        "low": close - np.abs(rng.normal(1.0, 0.3, n)),
        "close": close,
        "volume": np.full(n, 100.0),
        "session": ["london"] * n,
    })
    df = build_all_indicators(df, CFG)
    df = add_regime_indicators(df, CFG)
    labels = classify_regime_series(df, CFG)
    tail_labels = labels.iloc[-30:]
    assert (tail_labels == RegimeLabel.TREND_UP).sum() >= 10
    tail_labels = labels.iloc[-30:]
    assert (tail_labels == RegimeLabel.TREND_UP).sum() >= 10

def test_no_trade_on_flat_zero_volatility_series():
    """A perfectly flat price series has zero ATR -> must be classified no_trade, never a trend."""
    n = 250
    ts = np.arange(n) * 900
    flat_price = np.full(n, 2000.0)
    df = pd.DataFrame({
        "timestamp_utc": ts,
        "open": flat_price,
        "high": flat_price,
        "low": flat_price,
        "close": flat_price,
        "volume": np.full(n, 100.0),
        "session": ["asia"] * n,
    })
    df = build_all_indicators(df, CFG)
    df = add_regime_indicators(df, CFG)
    labels = classify_regime_series(df, CFG)
    assert (labels.iloc[-20:] == RegimeLabel.NO_TRADE).all()

def test_rule_based_adapter_matches_direct_call():
    df = _prepared_df(n=300)
    adapter = RuleBasedRegimeClassifier(CFG)
    direct_labels = classify_regime_series(df, CFG)
    adapter_labels = adapter.classify_series(df)
    assert (direct_labels == adapter_labels).all()

def test_ml_stub_raises_not_implemented():
    """ML regime classifier must fail loudly if used before training - never silently fallback."""
    stub = MLRegimeClassifierStub()
    df = _prepared_df(n=50)
    try:
        stub.classify(df.iloc[-1])
        assert False, "Expected NotImplementedError"
    except NotImplementedError:
        pass

```

#### FILE: requirements.txt

```

annotated-doc==0.0.4
annotated-types==0.8.0
anyio==4.14.2
certifi==2026.7.22
charset-normalizer==3.4.9
click==8.4.2
colorama==0.4.6
fastapi==0.140.0
h11==0.16.0
httpcore==1.0.9
httpx==0.28.1
idna==3.18
iniconfig==2.3.0
joblib==1.5.3
narwhals==2.24.0

```

```
numpy==2.5.1
packaging==26.2
pandas==3.0.5
pluggy==1.6.0
pydantic==2.13.4
pydantic_core==2.46.4
Pygments==2.20.0
pytest==9.1.1
python-dateutil==2.9.0.post0
python-telegram-bot==22.8
PyYAML==6.0.3
requests==2.34.2
scikit-learn==1.9.0
scipy==1.18.0
six==1.17.0
starlette==1.3.1
threadpoolctl==3.6.0
typing-inspection==0.4.2
typing_extensions==4.16.0
tzdata==2026.3
urllib3==2.7.0
uvicorn==0.51.0
xgboost==3.3.0
MetaTrader5==5.0.6070
```

```
FILE: scripts\__init__.py
```

```
# [EMPTY FILE]
```

```
FILE: scripts\backfill_data.py
```

```
"""
Backfill local FxPro MT5 candle history into the multi-asset SQLite database.

Examples:
    python -m scripts.backfill_data --all --timeframe M15 --start 2023-10-01 --end 2026-07-30
    python -m scripts.backfill_data --asset XAUUSD --timeframe M15 --start 2023-10-01 --end 2026-07-30
"""
import argparse
import logging
import os
import sys
from datetime import datetime, time, timezone

import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from config.loader import load_config
from data.mt5_provider import fetch_candles_range, shutdown_mt5
from data.storage import init_schema, upsert_candles

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
)
logger = logging.getLogger("backfill_data")

def _utc_bounds(start_date: str, end_date: str) -> tuple[datetime, datetime]:
    start = datetime.combine(
        datetime.strptime(start_date, "%Y-%m-%d").date(),
        time.min,
        tzinfo=timezone.utc,
    )
    end = datetime.combine(
        datetime.strptime(end_date, "%Y-%m-%d").date(),
        time.max,
        tzinfo=timezone.utc,
    )
    return start, end

def _session_label(timestamp: pd.Timestamp) -> str:
    weekday = timestamp.weekday()
    if weekday >= 5:
        return "weekend"
    hour = timestamp.hour
    if 0 <= hour < 8:
        return "asia"
    if 8 <= hour < 13:
        return "london"
    return "new_york"

def _to_storage_frame(df: pd.DataFrame) -> pd.DataFrame:
    frame = df.copy()
```

```

timestamp_column = next(
    (
        name
        for name in ("timestamp", "time", "datetime", "timestamp_utc")
        if name in frame.columns
    ),
    None,
)
if timestamp_column is None:
    raise ValueError(
        f"MT5 frame has no timestamp column. Columns: {list(frame.columns)}"
    )

timestamps = pd.to_datetime(frame[timestamp_column], utc=True)
epoch = pd.Timestamp("1970-01-01", tz="UTC")
frame["timestamp_utc"] = (
    (timestamps - epoch).dt.total_seconds().astype("int64")
)

if "volume" not in frame.columns:
    if "tick_volume" in frame.columns:
        frame["volume"] = frame["tick_volume"]
    elif "real_volume" in frame.columns:
        frame["volume"] = frame["real_volume"]
    else:
        frame["volume"] = 0.0

frame["session"] = timestamps.map(_session_label)

required = [
    "timestamp_utc",
    "open",
    "high",
    "low",
    "close",
    "volume",
    "session",
]
missing = set(required) - set(frame.columns)
if missing:
    raise ValueError(f"MT5 frame missing columns: {sorted(missing)}")

return (
    frame[required]
    .drop_duplicates(subset=["timestamp_utc"], keep="last")
    .sort_values("timestamp_utc")
    .reset_index(drop=True)
)

```

```

def main() -> None:
    parser = argparse.ArgumentParser(
        description="Backfill enabled assets from the locally running FxPro MT5 terminal."
    )
    target = parser.add_mutually_exclusive_group(required=True)
    target.add_argument("--asset", help="Internal asset key, for example XAUUSD")
    target.add_argument("--all", action="store_true", help="Backfill every enabled asset")
    parser.add_argument(
        "--timeframe",
        default=None,
        choices=["M1", "M5", "M15", "H1", "H4"],
        help="Defaults to config market_data.timeframe",
    )
    parser.add_argument("--start", required=True, help="YYYY-MM-DD, UTC")
    parser.add_argument("--end", required=True, help="YYYY-MM-DD, UTC")
    parser.add_argument(
        "--db-path",
        default="data/market_data_mt5.sqlite",
        help="SQLite destination path",
    )
    args = parser.parse_args()

    start_utc, end_utc = _utc_bounds(args.start, args.end)
    if end_utc <= start_utc:
        raise SystemExit("--end must be on or after --start.")

    cfg = load_config()
    market_data = cfg.get("market_data", {})
    if market_data.get("provider") != "mt5":
        raise SystemExit("config/config.yaml must specify market_data.provider: mt5")

    timeframe = args.timeframe or market_data.get("timeframe", "M15")
    assets = cfg.get("assets", {})

    if args.all:
        selected = {

```

```

        asset_key: asset_cfg
        for asset_key, asset_cfg in assets.items():
            if asset_cfg.get("enabled", False)
    }
else:
    if args.asset not in assets:
        raise SystemExit(f"Unknown asset key: {args.asset}")
    if not assets[args.asset].get("enabled", False):
        raise SystemExit(f"Asset is disabled: {args.asset}")
    selected = {args.asset: assets[args.asset]}

if not selected:
    raise SystemExit("No enabled assets selected.")

init_schema(args.db_path, [timeframe])

try:
    for asset_key, asset_cfg in selected.items():
        mt5_symbol = asset_cfg["mt5_symbol"]

        logger.info(
            "Fetching %s (%s), %s, %s through %s",
            asset_key,
            mt5_symbol,
            timeframe,
            args.start,
            args.end,
        )

        raw = fetch_candles_range(
            symbol=mt5_symbol,
            timeframe=timeframe,
            start_utc=start_utc,
            end_utc=end_utc,
        )

        stored = _to_storage_frame(raw)
        if stored.empty:
            raise RuntimeError(f"{asset_key}: MT5 returned no candles.")

        upsert_candles(args.db_path, timeframe, asset_key, stored)

        first = pd.to_datetime(stored["timestamp_utc"].iloc[0], unit="s", utc=True)
        last = pd.to_datetime(stored["timestamp_utc"].iloc[-1], unit="s", utc=True)

        logger.info(
            "%s: stored %d candles (%s through %s)",
            asset_key,
            len(stored),
            first.isoformat(),
            last.isoformat(),
        )
    finally:
        shutdown_mt5()

if __name__ == "__main__":
    main()

```

FILE: scripts\run\_backtest.py

```

import os
import sys
import argparse
import sqlite3
import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from config.loader import load_config
from features.indicators import build_all_indicators
from features.candle_anatomy import candle_anatomy
from regime.classifier import add_regime_indicators, classify_regime_series
from labeling.label_generator import generate_labels_from_config
from model.trainer import (
    build_training_matrix,
    train_model,
    calibrate_model,
    save_model,
)
from backtest.walk_forward import run_walk_forward

def load_asset_history(db_path: str, timeframe: str, asset_key: str) -> pd.DataFrame:
    table = f"ohlcv_{timeframe.lower()}"
    with sqlite3.connect(db_path) as conn:
        df = pd.read_sql_query(
            f"""

```

```

        SELECT
            timestamp_utc,
            open,
            high,
            low,
            close,
            volume,
            session
        FROM {table}
        WHERE symbol = ?
        ORDER BY timestamp_utc
    """
    conn,
    params=(asset_key,),
)
if df.empty:
    raise ValueError(f"No rows found for {asset_key} in {table}")

df["timestamp_utc"] = df["timestamp_utc"].astype("int64")
df["timestamp"] = pd.to_datetime(df["timestamp_utc"], unit="s", utc=True)
return df

def build_full_df(cfg, df):
    df = df.copy()
    df = build_all_indicators(df, cfg)
    df = candle_anatomy(df)
    df = add_regime_indicators(df, cfg)
    df["mtf_confluence_score"] = 0.0
    df["regime"] = classify_regime_series(df, cfg)
    df["label"] = generate_labels_from_config(df, cfg)
    return df

def strategy_fn_factory(cfg, model_path: str):
    def strategy_fn(train_df, test_df, cfg_inner):
        X, y, cols = build_training_matrix(train_df)
        if len(X) >= 30 and y.nunique() >= 2:
            base = train_model(X, y, cfg_inner)
            calibrated = calibrate_model(base, X, y, cfg_inner)
            save_model(calibrated, cols, model_path)

        from backtest.engine import EventDrivenBacktester
        from backtest.metrics import trades_to_dataframe, compute_metrics

        engine = EventDrivenBacktester(cfg_inner)
        trades = engine.run(test_df.reset_index(drop=True))
        trades_df = trades_to_dataframe(trades)
        return compute_metrics(trades_df)

    return strategy_fn

def main():
    parser = argparse.ArgumentParser(description="Run walk-forward backtest for one asset from SQLite.")
    parser.add_argument("--asset", required=True, help="Internal asset key, e.g. XAUUSD")
    parser.add_argument("--timeframe", default="M15")
    parser.add_argument("--db-path", default="data/market_data_mt5.sqlite")
    args = parser.parse_args()

    cfg = load_config()
    assets = cfg.get("assets", {})
    if args.asset not in assets:
        raise SystemExit(f"Unknown asset: {args.asset}")

    asset_cfg = assets[args.asset]
    model_path = asset_cfg["model_path"]

    raw = load_asset_history(args.db_path, args.timeframe, args.asset)
    df = build_full_df(cfg, raw)

    print(f"Loaded {len(df)} rows for {args.asset} from {args.db_path}")
    print(f"Training model output path: {model_path}")

    results = run_walk_forward(df, cfg, strategy_fn_factory(cfg, model_path))
    if not results:
        raise SystemExit("No walk-forward folds produced.")

    results_df = pd.DataFrame([r for r in results])
    os.makedirs("logs", exist_ok=True)
    results_df.to_csv(f"logs/backtest_{args.asset.lower()}.csv", index=False)

    print(f"Saved metrics to logs/backtest_{args.asset.lower()}.csv")
    print(f"Saved model to {model_path}")

if __name__ == "__main__":

```

```
main()
```

FILE: scripts\run\_scheduler.py

```
"""
Scheduler/worker loop: runs the realtime pipeline on every newly closed candle of
the primary labeling timeframe, logs every signal (regardless of alert outcome) to
SQLite, and sends a Telegram alert only if the ensemble + bot gating logic qualifies.

Design decision - candle-aligned scheduling, not fixed-interval polling:
Running strictly on a fixed interval (e.g. every 60s) would waste API calls and risk
scoring a candle mid-formation. Instead, this scheduler computes the exact UTC second
when the NEXT candle boundary closes (e.g. next M15 mark) and sleeps until shortly
after that boundary, guaranteeing every run scores a fully closed candle - consistent
with the "never score an in-progress candle" guarantee in realtime/pipeline.py.

Runs as a long-lived process (`python -m scripts.run_scheduler`), intended to be
supervised by systemd/Docker/pm2 in production - this module itself does not daemonize.
"""
import time
import logging
from datetime import datetime, timezone

from config.loader import load_config, get_env
from data.ingestion import TIMEFRAME_TO_SECONDS
from data.signal_log import init_schema, log_signal
from realtime.pipeline import RealtimePipeline
from alerts.telegram_bot import TelegramAlertBot

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
)
logger = logging.getLogger("scheduler")

def seconds_until_next_candle_close(timeframe: str, buffer_seconds: int = 5) -> float:
    """
    Computes how long to sleep so the NEXT wakeup happens `buffer_seconds` after the
    next candle boundary closes (giving the data vendor time to finalize the candle).
    """
    step = TIMEFRAME_TO_SECONDS[timeframe]
    now = time.time()
    next_boundary = ((int(now) // step) + 1) * step
    return max(0.0, (next_boundary - now) + buffer_seconds)

def run_once(pipeline: RealtimePipeline, bot: TelegramAlertBot, db_path: str, n_candles: int) -> dict:
    """
    Single pipeline pass: generate signal, log it unconditionally, send alert if qualified.
    Never raises out of this function during the main loop - errors are logged and
    the loop continues to the next scheduled candle rather than crashing the process.
    """
    signal = pipeline.generate_signal(n_candles=n_candles)
    alert_sent = bot.send_alert_if_qualified(signal)
    log_signal(db_path, signal, alert_sent)

    logger.info(
        "Signal generated: bias=%s confidence=%.3f regime=%s alert_sent=%s",
        signal["bias"], signal["confidence"], signal["regime"], alert_sent,
    )
    return signal

def main():
    cfg = load_config()
    timeframe = cfg["labeling"]["labeling_timeframe"]
    data_mode = get_env("DATA_MODE", default="mock")
    model_path = get_env("MODEL_PATH", default=None)
    db_path = get_env("SIGNAL_LOG_DB_PATH", default="data/signal_log.db")
    n_candles = int(get_env("PIPELINE_N_CANDLES", default="300"))

    init_schema(db_path)
    pipeline = RealtimePipeline(cfg=cfg, model_path=model_path, data_mode=data_mode)
    bot = TelegramAlertBot(cfg)

    logger.info("Scheduler starting. timeframe=%s data_mode=%s model_loaded=%s",
                timeframe, data_mode, pipeline._predictor is not None)

    while True:
        sleep_s = seconds_until_next_candle_close(timeframe)
        logger.info("Sleeping %.1fs until next %s candle close...", sleep_s, timeframe)
        time.sleep(sleep_s)

        try:
            run_once(pipeline, bot, db_path, n_candles)
        except Exception as e:
            logger.exception("Pipeline run failed, will retry on next scheduled candle: %s", e)
```

```
if __name__ == "__main__":
    main()
```

FILE: scripts\seed\_db.py

```
"""
One-time (or periodic) historical data seed script.
Pulls OHLCV candles for all configured timeframes from Twelve Data API
and writes them to the local SQLite database.

Usage:
    python scripts/seed_db.py                # seeds all timeframes, last 90 days
    python scripts/seed_db.py --days 180    # last 180 days
    python scripts/seed_db.py --timeframe M15 # single timeframe only

Requires env var: TWELVEDATA_API_KEY
"""
import os
import sys
import argparse
from datetime import datetime, timezone, timedelta

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from config.loader import load_config
from data.ingestion import backfill_historical
from data.storage import init_schema, upsert_candles
from data.session_tagger import tag_dataframe

def main():
    parser = argparse.ArgumentParser(description="Seed XAUUSD historical OHLCV data into SQLite.")
    parser.add_argument("--days", type=int, default=90, help="How many days back to seed (default: 90)")
    parser.add_argument("--timeframe", type=str, default=None, help="Single timeframe to seed (default: all)")
    args = parser.parse_args()

    cfg = load_config()
    db_path = cfg["general"]["db_path"]
    timeframes = [args.timeframe] if args.timeframe else cfg["timeframes"]
    sessions_cfg = cfg["sessions"]

    init_schema(db_path, timeframes)

    end_date = datetime.now(timezone.utc)
    start_date = end_date - timedelta(days=args.days)
    start_str = start_date.strftime("%Y-%m-%d")
    end_str = end_date.strftime("%Y-%m-%d")

    print(f"Seeding {len(timeframes)} timeframe(s) from {start_str} to {end_str} into {db_path}")

    for tf in timeframes:
        print(f" [{tf}] fetching...", end=" ", flush=True)
        try:
            df = backfill_historical(
                timeframe=tf,
                start_date=start_str,
                end_date=end_str,
                sessions_config=sessions_cfg,
            )
            if df.empty:
                print("no data returned - skipping")
                continue
            upsert_candles(db_path, tf, df)
            print(f"{len(df)} candles written")
        except Exception as e:
            print(f"ERROR: {e}")

    print("Seed complete.")

if __name__ == "__main__":
    main()
```

FILE: scripts\tests\\_\_init\_\_.py

```
# [EMPTY FILE]
```

FILE: scripts\tests\test\_scheduler.py

```
"""
Unit tests for scripts/run_scheduler.py - timing math and single-pass execution.
The infinite loop (main()) is NOT tested directly; run_once() and the sleep-time
calculation are the testable, deterministic units.
Run with: pytest scripts/tests/test_scheduler.py -v
"""
import os
```



```

import sys
import time
from unittest.mock import patch, MagicMock

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from scripts.run_scheduler import seconds_until_next_candle_close, run_once
from realtime.pipeline import RealtimePipeline
from alerts.telegram_bot import TelegramAlertBot

CFG = load_config()

def test_seconds_until_next_candle_close_is_positive_and_bounded():
    sleep_s = seconds_until_next_candle_close("M15", buffer_seconds=5)
    assert 0 < sleep_s <= 900 + 5

def test_seconds_until_next_candle_close_respects_buffer():
    """Sleep time must always include at least the buffer after the boundary."""
    sleep_s_no_buffer = seconds_until_next_candle_close("M15", buffer_seconds=0)
    sleep_s_with_buffer = seconds_until_next_candle_close("M15", buffer_seconds=10)
    assert sleep_s_with_buffer >= sleep_s_no_buffer

@patch("alerts.telegram_bot.requests.post")
def test_run_once_logs_signal_and_returns_dict(mock_post, tmp_path):
    mock_resp = MagicMock()
    mock_resp.raise_for_status = MagicMock()
    mock_post.return_value = mock_resp

    from data.signal_log import init_schema, read_signal_history
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)

    pipeline = RealtimePipeline(cfg=CFG, model_path=None, data_mode="mock")
    bot = TelegramAlertBot(CFG, bot_token="fake", chat_id="fake")

    result = run_once(pipeline, bot, db_path, n_candles=300)
    assert result["bias"] in ("long", "short", "no_trade")

    history = read_signal_history(db_path)
    assert len(history) == 1

@patch("alerts.telegram_bot.requests.post")
def test_run_once_does_not_alert_when_no_trade(mock_post, tmp_path):
    from data.signal_log import init_schema, read_signal_history
    db_path = str(tmp_path / "test.db")
    init_schema(db_path)

    pipeline = RealtimePipeline(cfg=CFG, model_path=None, data_mode="mock")
    bot = TelegramAlertBot(CFG, bot_token="fake", chat_id="fake")

    result = run_once(pipeline, bot, db_path, n_candles=300)
    if result["bias"] == "no_trade":
        assert not mock_post.called

```

FILE: scripts\train\_mt5.py

```

"""
Train one per-asset model from the MT5-backed SQLite database.

Example:
    python -m scripts.train_mt5 --symbol XAUUSD --db-path data/market_data_mt5.sqlite --output output/models/xauusd_direction_model.joblib
"""
import argparse
import os
import sys

import pandas as pd

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from config.loader import load_config
from data.storage import read_candles
from features.indicators import build_all_indicators
from features.candle_anatomy import candle_anatomy
from features.structure import detect_structure
from regime.classifier import add_regime_indicators, classify_regime_series
from labeling.label_generator import generate_labels_from_config
from model.trainer import (
    build_training_matrix,
    time_ordered_split,
    train_model,
    calibrate_model,
    save_model,

```

)

```
def build_full_df(df: pd.DataFrame, cfg: dict) -> pd.DataFrame:
    df = df.copy()
    df = build_all_indicators(df, cfg)
    df = candle_anatomy(df)
    df = detect_structure(df, lookback=cfg["features"]["structure_lookback"])
    df = add_regime_indicators(df, cfg)
    df["mtf_confluence_score"] = 0.0
    df["regime"] = classify_regime_series(df, cfg)
    df["label"] = generate_labels_from_config(df, cfg)
    return df

def main():
    parser = argparse.ArgumentParser(description="Train one MT5-backed model for one symbol.")
    parser.add_argument("--symbol", required=True, help="Internal asset key, e.g. XAUUSD")
    parser.add_argument("--db-path", default="data/market_data_mt5.sqlite")
    parser.add_argument("--timeframe", default="M15", choices=["M1", "M5", "M15", "H1", "H4"])
    parser.add_argument("--output", required=True, help="Destination .joblib path")
    args = parser.parse_args()

    cfg = load_config()

    raw = read_candles(args.db_path, args.timeframe, args.symbol)
    if raw.empty:
        raise SystemExit(f"No candles found for {args.symbol} in {args.db_path} timeframe={args.timeframe}")

    df = build_full_df(raw, cfg)

    X, y, cols = build_training_matrix(df)
    if len(X) < 500:
        raise SystemExit(f"Not enough labeled rows after feature/label prep for {args.symbol}: {len(X)}")
    if y.nunique() < 2:
        raise SystemExit(f"Need both classes present for {args.symbol}; got only one class.")

    train_ratio = cfg["model"].get("train_ratio", 0.8)
    X_train, X_test, y_train, y_test = time_ordered_split(X, y, train_ratio)

    base = train_model(X_train, y_train, cfg)
    calibrated = calibrate_model(base, X_train, y_train, cfg)
    save_model(calibrated, cols, args.output)

    print(f"symbol={args.symbol}")
    print(f"candles_raw={len(raw)}")
    print(f"rows_featured={len(df)}")
    print(f"rows_labeled_binary={len(X)}")
    print(f"train_rows={len(X_train)}")
    print(f"test_rows={len(X_test)}")
    print(f"class_counts={y.value_counts().to_dict()}")
    print(f"saved_model={args.output}")

if __name__ == "__main__":
    main()
```

FILE: tests\\_\_init\_\_.py

# [EMPTY FILE]

FILE: tests\integration\\_\_init\_\_.py

# [EMPTY FILE]